

ISTQB · CTFL

Tester Certificado Nivel Básico

v 4.0

GUÍA PERSONAL DE ESTUDIO

6 Capítulos · Glosario · Errores Comunes

Tabla de Contenidos

1.	Capítulo 1 — Fundamentos de las Pruebas
----	---

2.	Capítulo 2 — Pruebas a lo Largo del CVDS
----	--

3.	Capítulo 3 — Pruebas Estáticas
----	--------------------------------

4.	Capítulo 4 — Análisis y Diseño de Pruebas
----	---

5.	Capítulo 5 — Gestión de las Actividades de Prueba
----	---

6.	Capítulo 6 — Herramientas de Prueba
----	-------------------------------------

7.	Glosario
----	----------

8.	Errores Comunes
----	-----------------

Capítulo 1 - Fundamentos de las Pruebas

La idea principal: Las pruebas son mucho más que simplemente hacer clic y esperar que algo falle. Es una disciplina estructurada y profesional que ayuda a incorporar calidad al software — no solo a encontrar errores al final.

Niveles Cognitivos en Este Capítulo

El examen ISTQB clasifica cada objetivo de aprendizaje en uno de tres niveles cognitivos (basados en la Taxonomía de Bloom):

Nivel	Palabra clave	Lo que espera el examen
K1	Recordar	Recuperar o reconocer un término, concepto o hecho — sin necesidad de explicación
K2	Comprender	Explicar, diferenciar, comparar o dar ejemplos — ir más allá de la simple memorización
K3	Aplicar	Usar una técnica en un escenario concreto para derivar un resultado

Objetivos de Aprendizaje del Capítulo 1

OA	K	Tema
FL-1.1.1	K1	Identificar los objetivos típicos de las pruebas
FL-1.1.2	K2	Diferenciar las pruebas de la depuración
FL-1.2.1	K2	Ejemplificar por qué las pruebas son necesarias
FL-1.2.2	K1	Recordar la relación entre las pruebas y el aseguramiento de la calidad
FL-1.2.3	K2	Distinguir entre error, defecto, fallo y causa raíz
FL-1.3.1	K2	Explicar los siete principios de las pruebas
FL-1.4.1	K2	Resumir las diferentes actividades y tareas de prueba
FL-1.4.2	K2	Explicar el impacto del contexto en el proceso de prueba
FL-1.4.3	K2	Diferenciar los productos de trabajo que apoyan las actividades de prueba
FL-1.4.4	K2	Explicar el valor de mantener la trazabilidad entre la base de prueba y los productos de trabajo de prueba
FL-1.5.1	K2	Explicar el rol de la psicología humana en las pruebas
FL-1.5.2	K1	Identificar las habilidades requeridas para las pruebas
FL-1.5.3	K2	Distinguir la mentalidad requerida para las pruebas de la mentalidad requerida para el desarrollo

Consejo para el examen: El Capítulo 1 es casi completamente K2 — enfócate en poder *explicar* y *distinguir*, no solo nombrar. Los únicos ítems K1 son FL-1.1.1, FL-1.2.2 y FL-1.5.2. No hay K3 en este capítulo.

¿Qué son realmente las Pruebas? — Comprender (K2)

Las pruebas de software son un conjunto de actividades para **descubrir defectos**, **evaluar la calidad** y **proporcionar información** para que las partes interesadas puedan tomar decisiones informadas. Las pruebas tienen siete objetivos oficiales (FL-1.1.1 — K1, conócelos todos):

1. Evaluar los productos de trabajo — requisitos, historias de usuario, diseños, código
2. Provocar fallos y encontrar defectos

3. Garantizar la cobertura requerida del objeto de prueba
4. Reducir el riesgo de calidad inadecuada del software
5. Verificar que los requisitos especificados se han cumplido
6. Verificar el cumplimiento de los requisitos contractuales y legales
7. Proporcionar información a las partes interesadas para permitir decisiones informadas

Pero esto es lo que mucha gente entiende mal sobre las pruebas:

- Las pruebas **no son solo** encontrar errores — también generan confianza en que el software funciona
- Las pruebas **no son solo** ejecutar el software — incluyen también la revisión de documentos, diseños y requisitos (pruebas estáticas)
- Las pruebas involucran **tanto la verificación** ("¿estamos construyendo bien el producto?") como la **validación** ("¿estamos construyendo el producto correcto?")

Pruebas vs. Depuración — ¡no son lo mismo! — 💡 Comprender (K2)

Actividad	Quién la realiza	Qué hace
Pruebas	Tester	Encuentra fallos y defectos
Depuración	Desarrolladores	Encuentra la causa, la corrige, elimina el defecto

Las pruebas provocan el fallo → la depuración encuentra y corrige la causa raíz.

¿Por Qué son Necesarias las Pruebas? — 💡 Comprender (K2)

Los defectos en el software ocurren porque los humanos cometen errores. Esos errores crean defectos, y los defectos — cuando se ejecutan — causan fallos. Y los fallos pueden costar dinero, reputación o incluso vidas.

La cadena error → defecto → fallo — 💡 Comprender (K2)

- **Error:** Un equivocación humana (p. ej., un desarrollador malinterpreta un requisito)
- **Defecto:** La falla que el error introduce en el código o documento
- **Fallo:** Lo que ocurre cuando ese defecto se ejecuta y el sistema se comporta incorrectamente
- **Causa raíz:** La razón subyacente por la que ocurrió el error en primer lugar

No todos los defectos causan fallos — algunos solo fallan bajo condiciones específicas, y algunos pueden no causar nunca un fallo.

Pruebas vs. Aseguramiento de la Calidad (QA) — ¡tampoco son lo mismo! — 🧠

Recordar (K1)

- Las pruebas son **orientadas al producto** y **correctivas** — encuentran defectos en lo que se construyó
- QA es **orientado al proceso** y **preventivo** — mejora el proceso para evitar que ocurran defectos

Piensa en QA como establecer un buen proceso en la cocina, y en las pruebas como probar el plato para verificar que salió bien.

Los 7 Principios de las Pruebas — Comprender (K2)

Estos son la base de todo en ISTQB. Conócelos a la perfección.

#	Principio	Lo que significa en la práctica
1	Las pruebas muestran la presencia de defectos, no su ausencia	Pasar todas las pruebas no significa que el software esté libre de errores
2	Las pruebas exhaustivas son imposibles	No puedes probar todo — usa el riesgo y las prioridades para enfocarte
3	Las pruebas tempranas ahorran tiempo y dinero	Cuanto más tarde se encuentra un defecto, más caro es corregirlo
4	Los defectos se agrupan	Un pequeño número de módulos suele contener la mayoría de los defectos (principio de Pareto)
5	Las pruebas se desgastan	Ejecutar las mismas pruebas repetidamente encuentra menos defectos nuevos — varíalas y actualízalas
6	Las pruebas dependen del contexto	Probar una aplicación bancaria es muy diferente a probar un juego
7	La ausencia de defectos es una falacia	Un sistema sin errores que no cumple las necesidades del usuario sigue siendo un fracaso

Actividades y Tareas de Prueba — Comprender (K2)

Un proceso de prueba está compuesto por estas actividades — con frecuencia ocurren de forma iterativa, no estrictamente en secuencia:

1. **Planificación de pruebas** — Definir objetivos y enfoque
2. **Monitoreo y control de pruebas** — Seguir el progreso; tomar acciones correctivas si es necesario
3. **Análisis de pruebas** — ¿*Qué* probar? Analizar la base de prueba para identificar las condiciones de prueba
4. **Diseño de pruebas** — ¿*Cómo* probar? Convertir las condiciones en casos de prueba y definir los datos de prueba
5. **Implementación de pruebas** — Preparar todo lo necesario: scripts de prueba, suites de prueba, configuración del entorno
6. **Ejecución de pruebas** — Ejecutar las pruebas, registrar resultados, reportar anomalías
7. **Finalización de pruebas** — Concluir: archivar los productos de trabajo de prueba, escribir el informe de finalización, capturar las lecciones aprendidas

Productos de Trabajo de Prueba — lo que produce cada actividad — 💡 Comprender (K2)

Actividad	Ejemplos de salida
Planificación	Plan de pruebas, registro de riesgos
Monitoreo y control	Informes de progreso
Análisis	Condiciones de prueba, informes de defectos
Diseño	Casos de prueba, guías de prueba, requisitos de datos de prueba
Implementación	Scripts de prueba, suites de prueba, calendario de ejecución de pruebas
Ejecución	Registros de prueba, informes de defectos
Finalización	Informe de finalización de pruebas, lecciones aprendidas

Trazabilidad — 💡 Comprender (K2)

Una buena trazabilidad vincula los casos de prueba con los requisitos y los riesgos. Esto ayuda a: - Medir la cobertura - Analizar el impacto de los cambios - Apoyar auditorías e informes de gobernanza

Roles en las Pruebas — 💡 Comprender (K2)

Existen dos roles principales:

- **Rol de gestión de pruebas** — Se enfoca en la planificación, el monitoreo, el control y la finalización. Responsable del proceso de prueba y del liderazgo del equipo.
- **Rol de pruebas** — Se enfoca en el lado técnico: análisis, diseño, implementación y ejecución.

Una persona puede desempeñar ambos roles al mismo tiempo, dependiendo del contexto del proyecto.

Habilidades Esenciales para los Testers — 🧠 Recordar (K1)

Los buenos testers necesitan una combinación de: - **Conocimiento de pruebas** — saber cómo aplicar técnicas de manera efectiva - **Pensamiento analítico y crítico** — para encontrar lo que otros pasan por alto - **Buena comunicación** — los testers a menudo dan malas noticias; hacerlo de manera constructiva es importante - **Conocimiento del dominio** — comprender el contexto de negocio - **Conocimiento técnico** — usar herramientas de manera eficiente

Enfoque de Equipo Completo — 💡 Comprender (K2)

Todos en el equipo son responsables de la calidad — no solo los testers. Los testers colaboran estrechamente con los desarrolladores y los representantes del negocio. Esto proviene de Extreme Programming (XP).

Excepción: en sistemas críticos para la seguridad, puede ser necesario un mayor nivel de independencia de las pruebas.

Independencia de las Pruebas — 💡 Comprender (K2)

Los testers independientes encuentran defectos diferentes a los del autor porque tienen sesgos cognitivos distintos. Niveles de independencia: 1. El autor prueba su propio trabajo (menor independencia) 2. Pares del mismo equipo 3. Testers de fuera del equipo (alta independencia) 4. Testers de fuera de la organización (muy alta independencia)

Beneficio: Una perspectiva diferente detecta más defectos. **Inconveniente:** Riesgo de aislamiento, problemas de comunicación, desarrolladores que pierden el sentido de responsabilidad por la calidad, o la creación de una mentalidad de "nosotros contra ellos" entre testers y desarrolladores.

Hoja de Referencia de Términos Clave — 🧠 Recordar (K1)

Término	Definición
Error	Una equivocación humana
Defecto	Una falla introducida por un error
Fallo	Comportamiento incorrecto causado por la ejecución de un defecto
Causa raíz	La razón fundamental por la que existe un defecto
Base de prueba	Todo lo utilizado para derivar casos de prueba (requisitos, diseños, código, etc.)
Objeto de prueba	El elemento que se está probando
Productos de trabajo de prueba	Todos los artefactos producidos por las actividades de prueba
Verificación	¿Estamos construyendo bien el producto?
Validación	¿Estamos construyendo el producto correcto?

Preguntas de Práctica

- ¿Cuál es la diferencia entre error, defecto y fallo? Da un ejemplo de cada uno.
- Lista los 7 principios de las pruebas y explica dos de ellos con tus propias palabras.
- ¿Cuál es la diferencia entre las pruebas y el Aseguramiento de la Calidad (QA)?
- ¿Cuál es la diferencia entre verificación y validación?
- ¿Por qué las pruebas exhaustivas son imposibles?
- ¿Cuáles son los dos roles principales en las pruebas y en qué se enfoca cada uno?
- ¿Cuáles son los beneficios e inconvenientes de la independencia de las pruebas?
- ¿En qué ayuda la trazabilidad dentro del proceso de prueba?

Capítulo 2 - Pruebas a lo Largo del Ciclo de Vida del Desarrollo de Software

La idea principal: Las pruebas no ocurren al final. Viven dentro de cada fase del desarrollo, se adaptan al modelo que el equipo utilice y cambian de forma según el contexto.

Niveles Cognitivos en Este Capítulo

K1 = Recordar · K2 = Comprender · K3 = Aplicar — consulta el Capítulo 1 para la leyenda completa.

Objetivos de Aprendizaje del Capítulo 2

OA	K	Tema
FL-2.1.1	K2	Explicar el impacto del modelo de CVDS elegido en las pruebas
FL-2.1.2	K1	Recordar las buenas prácticas de prueba que aplican a cualquier modelo de CVDS
FL-2.1.3	K1	Recordar ejemplos de TDD, ATDD y BDD
FL-2.1.4	K2	Resumir los beneficios de DevOps para las pruebas
FL-2.1.5	K2	Explicar el enfoque de desplazamiento hacia la izquierda
FL-2.1.6	K2	Explicar cómo las retrospectivas pueden usarse para la mejora del proceso en las pruebas
FL-2.2.1	K2	Distinguir los diferentes niveles de prueba
FL-2.2.2	K2	Distinguir los diferentes tipos de prueba
FL-2.2.3	K2	Distinguir las pruebas de confirmación de las pruebas de regresión
FL-2.3.1	K2	Resumir los disparadores de las pruebas de mantenimiento
FL-2.3.2	K2	Explicar el rol del análisis de impacto en las pruebas de mantenimiento

Consejo para el examen: No hay K3 en este capítulo. Los ítems K1 son FL-2.1.2 y FL-2.1.3 — solo memorizar, sin necesidad de explicación. Todo lo demás requiere que *expliques* o *distingas*.

2.1 Las Pruebas en el Contexto de un CVDS — 💡 Comprender (K2)

Un modelo de CVDS define cómo se relacionan entre sí las fases y actividades del desarrollo. La elección del modelo impacta directamente en **cómo** se realizan las pruebas — no en si se realizan o no.

Las tres categorías principales de modelos de CVDS — 💡 Comprender (K2)

Modelo	Ejemplos	Cómo encajan las pruebas
Secuencial	Cascada, Modelo en V	Las pruebas comienzan después del desarrollo; el Modelo en V mapea cada etapa de desarrollo a un nivel de prueba
Iterativo	Espiral, Prototipado	Las pruebas ocurren dentro de cada ciclo repetido
Incremental	Proceso Unificado	Cada pequeño incremento se prueba antes de pasar al siguiente

En la práctica, muchos proyectos reales combinan modelos — p. ej., Agile usa enfoques tanto iterativos como incrementales. Las prácticas ágiles como Scrum, Kanban, TDD, BDD y XP son métodos que operan *dentro* de estos tipos de modelos.

El CVDS impacta las pruebas de 5 maneras — 💡 Comprender (K2)

1. Alcance y oportunidad de las actividades de prueba (cuándo y qué se prueba)
2. Nivel de detalle de la documentación de prueba
3. Elección de técnicas y enfoque de prueba
4. Extensión de la automatización de pruebas
5. Roles y responsabilidades de los testers

Buenas prácticas de prueba — aplican a CUALQUIER modelo de CVDS — 🧠 Recordar (K1)

- Cada actividad de desarrollo tiene una actividad de prueba correspondiente
- Los diferentes niveles de prueba tienen objetivos de prueba específicos y distintos
- El análisis y diseño de pruebas comienzan durante la fase de desarrollo correspondiente
- Los testers revisan los productos de trabajo tan pronto como los borradores estén disponibles

Las Pruebas como Motor: TDD, ATDD y BDD — 🧠 Recordar (K1)

Estos enfoques invierten el orden tradicional — las pruebas se escriben antes que el código.

Enfoque	Quién lo usa	Cómo funciona
TDD (Desarrollo Guiado por Pruebas)	Desarrolladores	Escribir una prueba fallida → escribir código para pasarla → refactorizar
ATDD (Desarrollo Guiado por Pruebas de Aceptación)	Todo el equipo	Derivar pruebas de los criterios de aceptación antes de construir la funcionalidad
BDD (Desarrollo Guiado por Comportamiento)	Todo el equipo	Escribir pruebas en lenguaje natural usando el formato Dado/Cuando/Entonces

Los tres implementan el principio de pruebas tempranas y apoyan el desplazamiento hacia la izquierda.

Consejo para el examen (K1): Para FL-2.1.3, solo necesitas recordar quién usa cada enfoque y su idea central. Los detalles de "cómo funciona" en la tabla son para contexto — el examen evalúa reconocimiento, no reproducción del procedimiento.

DevOps y las Pruebas — 💡 Comprender (K2)

DevOps conecta desarrollo, pruebas y operaciones para entregar software de alta calidad más rápido.

Beneficios para las pruebas: - Retroalimentación rápida sobre la calidad del código a través de pipelines de CI/CD - Pruebas de regresión automatizadas que se ejecutan en cada confirmación de código - Mayor visibilidad sobre la calidad no funcional (rendimiento, confiabilidad) - Reduce las pruebas manuales repetitivas mediante la automatización

Alcance del examen: Los *beneficios* de DevOps para las pruebas se evalúan directamente (FL-2.1.4, K2). Los riesgos a continuación son contexto de apoyo no cubierto por un OA específico — compréndalos pero no los memorices.

Riesgos y desafíos: - El pipeline de DevOps debe definirse y mantenerse - La automatización de pruebas requiere recursos adicionales - Las herramientas de CI/CD deben introducirse y mantenerse actualizadas

Incluso con una fuerte automatización, las pruebas manuales — especialmente desde la perspectiva del usuario — siguen siendo necesarias.

Desplazamiento hacia la Izquierda — 💡 Comprender (K2)

El desplazamiento hacia la izquierda significa **probar más temprano en el CVDS** — no esperar hasta que el código esté completo.

Buenas prácticas para lograr el desplazamiento hacia la izquierda: - Revisar las especificaciones desde la perspectiva del tester (detectar ambigüedades temprano) - Escribir casos de prueba antes de que el código esté escrito - Usar CI/CD para obtener retroalimentación rápida en cada confirmación de código - Realizar análisis estático antes de las pruebas dinámicas - Comenzar las pruebas no funcionales a nivel de componente, cuando sea posible

El desplazamiento hacia la izquierda puede aumentar el esfuerzo y el costo iniciales, pero ahorra significativamente más en fases posteriores.

Retrospectivas y Mejora del Proceso — 💡 Comprender (K2)

Las retrospectivas se realizan al final de una iteración, proyecto o hito de lanzamiento. Los participantes (no solo testers — también desarrolladores, arquitectos, propietarios de producto, analistas de negocio) debaten: - ¿Qué salió bien y debe conservarse? - ¿Qué no salió bien y podría mejorarse? - ¿Cómo incorporar las mejoras en el futuro?

Beneficios para las pruebas: - Mayor efectividad y eficiencia de las pruebas - Mejor calidad de los productos de trabajo de prueba - Integración del equipo y aprendizaje - Mejor cooperación entre desarrollo y pruebas

2.2 Niveles de Prueba y Tipos de Prueba — 💡 Comprender (K2)

Niveles de Prueba — los cinco niveles — 💡 Comprender (K2)

Cada nivel tiene un enfoque, una base de prueba y un conjunto de objetivos diferentes.

Nivel	Enfoque	Típicamente realizado por
Prueba de componente (unitaria)	Componentes individuales en aislamiento	Desarrolladores
Prueba de integración de componentes	Interfaces e interacciones entre componentes	Desarrolladores / Testers
Prueba del sistema	Comportamiento de extremo a extremo del sistema completo	Testers independientes
Prueba de integración del sistema	Interfaces entre el sistema y sistemas/servicios externos	Testers
Prueba de aceptación	Validación; demostrar preparación para el despliegue	Negocio / Usuarios finales

Tipos de Prueba de Aceptación — 💡 Comprender (K2)

- **UAT** (Pruebas de Aceptación de Usuario): Los usuarios reales validan que el sistema cumple sus necesidades
- **OAT** (Pruebas de Aceptación Operacional): Preparación operativa — respaldo, recuperación, seguridad
- **Contractual/Regulatoria**: Verifica el cumplimiento de contratos o regulaciones
- **Pruebas Alfa**: Usuarios internos prueban en las instalaciones del desarrollador
- **Pruebas Beta**: Usuarios externos prueban en su propio entorno

Tipos de Prueba — qué aspecto se está probando — 💡 Comprender (K2)

Tipo	Qué verifica
Prueba funcional	Lo que hace el sistema — completitud funcional, corrección, idoneidad
Prueba no funcional	Qué tan bien se comporta el sistema — rendimiento, seguridad, usabilidad, confiabilidad, mantenibilidad, portabilidad, seguridad funcional
Prueba de caja negra	Comportamiento basado en especificaciones, sin conocimiento de la estructura interna
Prueba de caja blanca	Estructura interna — rutas de código, arquitectura, flujos de datos

Los cuatro tipos de prueba pueden aplicarse en **cualquier** nivel de prueba.

Nota: "Caja negra" y "caja blanca" aparecen tanto en el Capítulo 2 (como *tipos* de prueba) como en el Capítulo 4 (como categorías de *técnicas* de prueba). En el Capítulo 2 clasifican *en qué se basan las pruebas* (especificación vs. estructura); en el Capítulo 4 clasifican *cómo se derivan los casos de prueba*. Mismos términos, significado relacionado pero distinto — no dejes que te confunda.

Pruebas Relacionadas con Cambios — 💡 Comprender (K2)

- **Pruebas de confirmación:** Verifican que un defecto específico ha sido corregido
- **Pruebas de regresión:** Garantizan que los cambios no han roto la funcionalidad existente

Las pruebas de regresión crecen con cada versión — son una candidata fuerte para la automatización.

2.3 Pruebas de Mantenimiento — 💡 Comprender (K2)

Las pruebas de mantenimiento se desencadenan cuando un sistema que ya está en producción necesita cambiar.

Disparadores (categorías de la v4) — 💡 Comprender (K2)

Disparador	Ejemplos
Modificaciones	Mejoras planificadas, cambios correctivos (corrección de errores), correcciones urgentes
Actualizaciones o migraciones	Migración a una nueva plataforma, sistema operativo o entorno
Retiro	Fin de vida útil — pruebas de archivo y recuperación de datos

El alcance depende de — 💡 Comprender (K2)

- El grado de riesgo del cambio
- El tamaño del sistema existente
- El tamaño del cambio

Análisis de Impacto — 💡 Comprender (K2)

Antes de las pruebas, el **análisis de impacto** determina qué partes del sistema podrían verse afectadas por el cambio. Esto ayuda a definir el alcance de las pruebas de regresión y depende de una buena trazabilidad.

Término	Definición
CVDS	Ciclo de Vida del Desarrollo de Software — el proceso para planificar, crear, probar y entregar software
Nivel de prueba	Un grupo de actividades de prueba organizadas juntas (componente, integración, sistema, aceptación)
Tipo de prueba	Clasificación por lo que se está probando (funcional, no funcional, caja negra, caja blanca)
Desplazamiento hacia la izquierda	Probar más temprano en el ciclo de vida para detectar defectos antes
TDD	Las pruebas se escriben antes del código para guiar el desarrollo
BDD	Las pruebas se escriben en formato de lenguaje natural Dado/Cuando/Entonces
Pruebas de regresión	Re-ejecutar pruebas para detectar efectos secundarios no deseados de un cambio
Pruebas de confirmación	Re-probar después de una corrección para verificar que el defecto está resuelto
Análisis de impacto	Evaluar qué partes del sistema se ven afectadas por un cambio

Preguntas de Práctica

- ¿Cuáles son las tres categorías principales de modelos de CVDS? Da un ejemplo de cada una.
- ¿Cómo impacta la elección del modelo de CVDS en las pruebas? Nombra al menos tres formas.
- ¿Cuál es la diferencia entre TDD, ATDD y BDD?
- ¿Cuáles son los beneficios y riesgos de DevOps para las pruebas?
- ¿Qué significa "desplazamiento hacia la izquierda" y cómo puedes lograrlo?
- ¿Cuál es la diferencia entre la prueba de componente y la prueba de integración de componentes?
- ¿Cuál es la diferencia entre las pruebas de confirmación y las pruebas de regresión?
- ¿Cuáles son los tres disparadores para las pruebas de mantenimiento en la v4?
- ¿Por qué las pruebas de regresión son una buena candidata para la automatización?
- ¿Cuál es el propósito del análisis de impacto en las pruebas de mantenimiento?

Capítulo 3 - Pruebas Estáticas

La idea central: No es necesario ejecutar el software para encontrar problemas en él. Revisar documentos, diseños y código antes de la ejecución suele detectar defectos de forma más temprana, más económica y en lugares que las pruebas dinámicas simplemente no pueden alcanzar.

Niveles Cognitivos en Este Capítulo

K1 = Recordar · K2 = Comprender · K3 = Aplicar — ver Capítulo 1 para la leyenda completa.

Objetivos de Aprendizaje del Capítulo 3

OA	K	Tema
FL-3.1.1	K1	Recordar los productos de trabajo que pueden examinarse mediante pruebas estáticas
FL-3.1.2	K2	Explicar el valor de las pruebas estáticas
FL-3.1.3	K2	Comparar las pruebas estáticas y las pruebas dinámicas
FL-3.2.1	K1	Recordar las actividades, roles y responsabilidades de una revisión formal
FL-3.2.2	K2	Explicar en qué se diferencian los distintos tipos de revisión entre sí
FL-3.2.3	K1	Recordar los factores para una revisión exitosa

Consejo para el examen: No hay K3 en este capítulo. Tres elementos K1 (FL-3.1.1, FL-3.2.1, FL-3.2.3) — pura memorización. FL-3.1.3 y FL-3.2.2 requieren *comparar* y *explicar diferencias* — K2 clásico.

3.1 Fundamentos de las Pruebas Estáticas — 💡 Comprender (K2)

Las pruebas estáticas consisten en evaluar un producto de trabajo **sin ejecutarlo**. Pueden realizarse: - **Manualmente** — mediante revisiones (recorridos, inspecciones, etc.) - **Con herramientas** — mediante análisis estático (p. ej., linters, analizadores de código integrados en pipelines de CI)

Se aplican tanto a la **verificación** como a la **validación**, y sus objetivos incluyen mejorar la calidad, detectar defectos y evaluar la legibilidad, completitud, corrección, capacidad de prueba y consistencia.

¿Qué Puede Examinarse con las Pruebas Estáticas? — 🧠 Recordar (K1)

Prácticamente **cualquier producto de trabajo** — siempre que pueda leerse y comprenderse:

- Documentos de especificación de requisitos
- Código fuente
- Planes de prueba y casos de prueba
- Elementos del backlog del producto, actas de prueba
- Documentación del proyecto, contratos y modelos

Para el análisis estático específicamente, el producto de trabajo necesita una sintaxis formal (p. ej., código, modelos o texto estructurado).

NO es apropiado para las pruebas estáticas: productos de trabajo difíciles de interpretar por personas o que no pueden ser analizados por herramientas por razones legales (p. ej., código ejecutable de terceros).

¿Por Qué Hacerlo? El Valor de las Pruebas Estáticas —💡 Comprender (K2)

Las pruebas estáticas son poderosas porque:

1. **Detectan defectos tempranamente** — antes de que queden incorporados en productos de trabajo posteriores y resulten costosos de corregir
2. **Encuentran lo que las pruebas dinámicas no pueden** — como código inalcanzable, fallos de diseño o defectos en documentos no ejecutables
3. **Generan comprensión compartida** — revisar los requisitos en conjunto alinea al equipo desde el principio
4. **Son rentables** — el costo inicial de las revisiones es mucho menor que el costo de encontrar esos mismos defectos tarde en el proyecto

Defectos más fáciles y económicos de encontrar mediante pruebas estáticas —💡 Comprender (K2)

Categoría	Ejemplos
Defectos de requisitos	Inconsistencias, ambigüedades, contradicciones, omisiones, duplicaciones
Defectos de diseño	Estructuras de base de datos ineficientes, modularización deficiente
Defectos de codificación	Valores no definidos, variables no declaradas, código inalcanzable/duplicado, complejidad excesiva
Desviaciones de estándares	Violaciones de convenciones de nomenclatura, violaciones de estándares de codificación
Defectos de interfaz	Número, tipo u orden de parámetros incompatible
Vulnerabilidades de seguridad	Desbordamientos de búfer
Brechas en la base de pruebas	Pruebas faltantes para un criterio de aceptación

Comprender (K2)

Aspecto	Pruebas Estáticas	Pruebas Dinámicas
¿Requiere ejecución?	No	Sí
Cómo se encuentran los defectos	Directamente	Mediante fallos → análisis de causa raíz
Aplicable a	Productos de trabajo ejecutables Y no ejecutables	Solo productos de trabajo ejecutables
Caminos de código poco frecuentes	Más fácil de detectar	Difícil de alcanzar
Características de calidad	Mantenibilidad, legibilidad	Rendimiento, fiabilidad

Piénsalo así: **las pruebas estáticas leen la receta**, mientras que **las pruebas dinámicas prueban el plato**.

3.2 Retroalimentación y Proceso de Revisión — Comprender (K2)

Por Qué es Importante la Retroalimentación Temprana y Frecuente de los Interesados — Comprender (K2)

Si los interesados no participan con regularidad durante el desarrollo, el producto puede alejarse de su visión original. Las consecuencias: - Retrabajo costoso - Incumplimiento de plazos - Disputas sobre responsabilidades - Fracaso del proyecto

La retroalimentación frecuente a lo largo del SDLC: - Previene malentendidos sobre los requisitos desde el principio - Garantiza que los cambios se comprendan e implementen antes de que resulten costosos - Mantiene al equipo enfocado en lo que aporta mayor valor - Aborda los riesgos de forma proactiva

El Proceso de Revisión — 5 Actividades — Recordar (K1)

Basado en el estándar **ISO/IEC 20246**:

Paso	Qué ocurre
1. Planificación	Definir el alcance, propósito, criterios de salida, esfuerzo y plazos
2. Inicio de la revisión	Garantizar que todos los participantes tengan acceso, comprendan su rol y estén preparados
3. Revisión individual	Cada revisor evalúa independientemente el producto de trabajo; registra anomalías, recomendaciones y preguntas
4. Comunicación y análisis	Las anomalías se discuten en una reunión de revisión; se toman decisiones sobre el estado, la titularidad y las acciones de seguimiento
5. Corrección e informe	El autor corrige los defectos encontrados; el líder de revisión redacta el informe de revisión; el producto de trabajo se acepta una vez cumplidos los criterios de salida

Roles en una Revisión — ¿Quién Hace Qué? — 🧠 Recordar (K1)

Rol	Responsabilidad
Dirección	Decide qué se revisa; proporciona los recursos (personas, tiempo)
Autor	Crea y posteriormente corrige el producto de trabajo bajo revisión
Moderador (facilitador)	Dirige la reunión de revisión; gestiona la mediación, el tiempo y garantiza un entorno seguro
Escriba (registrador)	Registra anomalías, decisiones y nuevos hallazgos durante la reunión
Revisor	Realiza la revisión en sí; puede ser un miembro del equipo, un experto en la materia o cualquier interesado
Líder de revisión	Es responsable general de la revisión; decide quién participa, cuándo y dónde

Truco de memoria: Piensa en una revisión como una reunión estructurada con un **jefe** (dirección), un **creador** (autor), un **árbitro** (moderador), un **anotador** (escriba), **evaluadores** (revisores) y un **organizador** (líder de revisión).

Tipo	Formalidad	Dirigida por	Características clave
Revisión informal	Mínima	Cualquiera	Sin proceso definido; sin resultado formal; solo encontrar anomalías
Recorrido	Baja–Media	Autor	El autor guía al equipo a través del producto de trabajo; educa, logra consenso y detecta anomalías; la revisión individual previa es opcional; el escriba es obligatorio
Revisión técnica	Media–Alta	Moderador capacitado o líder entre pares	Revisores técnicamente cualificados; enfoque en el consenso sobre problemas técnicos y la detección de anomalías
Inspección	Máxima	Moderador (NO el autor)	Sigue el proceso completo definido; la detección máxima de anomalías es el objetivo principal; se recopilan métricas; el autor no puede ser líder de revisión ni escriba

Punto clave para el examen: En una inspección, el **autor no puede actuar como líder de revisión ni escriba** — esto es exclusivo de este tipo de revisión.

Factores de Éxito para las Revisiones — Recordar (K1)

Una revisión tiene éxito cuando:

- Los objetivos son **claros y medibles** — la evaluación de los participantes **nunca** es un objetivo
- Se elige el **tipo de revisión adecuado** para el producto de trabajo y el contexto
- Las revisiones se realizan en **fragmentos pequeños** para que los revisores mantengan el enfoque
- Se da **retroalimentación** a los interesados y autores para que puedan mejorar
- Los participantes tienen **tiempo de preparación adecuado**
- La **dirección apoya** el proceso de revisión
- Las revisiones forman parte de la **cultura de la organización**
- Todos los participantes reciben **formación adecuada**
- Las reuniones están **facilitadas de manera efectiva**

Término	Definición
Pruebas estáticas	Evaluación de un producto de trabajo sin ejecutarlo
Pruebas dinámicas	Pruebas mediante la ejecución del software
Análisis estático	Evaluación asistida por herramientas contra una sintaxis formal
Revisión	Evaluación manual de un producto de trabajo por una o más personas
Inspección	Tipo de revisión más formal; sigue un proceso completo definido; el autor no puede ser líder ni escriba
Revisión técnica	Revisión dirigida por un moderador con revisores técnicamente cualificados
Recorrido	Revisión dirigida por el autor
Revisión informal	Revisión sin proceso definido ni resultado formal
Anomalía	Cualquier resultado que difiere de lo esperado; identificada durante una revisión
Moderador	Garantiza el funcionamiento efectivo de una reunión de revisión

Preguntas de Práctica

- ¿Qué son las pruebas estáticas y en qué se diferencian de las pruebas dinámicas?
- ¿Qué tipos de productos de trabajo pueden examinarse mediante pruebas estáticas?
- Menciona tres tipos de defectos que son más fáciles de encontrar mediante pruebas estáticas.
- ¿Cuáles son las cinco actividades del proceso de revisión?
- ¿Cuáles son los seis roles principales en una revisión? ¿Qué hace cada uno?
- ¿Cuál es el tipo de revisión más formal y qué lo hace único?
- ¿Cuál es la diferencia principal entre un recorrido y una revisión técnica?
- ¿Por qué el autor no puede ser líder de revisión ni escriba en una inspección?
- Menciona cuatro factores de éxito para las revisiones.
- ¿Por qué la evaluación de los participantes nunca debe ser un objetivo de una revisión?

Capítulo 4 - Análisis y Diseño de Pruebas

La idea central: ¿Cómo decides *qué* probar y *cómo* probarlo sin simplemente adivinar? Las técnicas de prueba te ofrecen una forma sistemática y repetible de diseñar casos de prueba que cubran lo que importa — basándose en especificaciones, estructura del código o tu propia experiencia.

Niveles Cognitivos en Este Capítulo

K1 = Recordar · K2 = Comprender · K3 = Aplicar — ver Capítulo 1 para la leyenda completa.

Objetivos de Aprendizaje del Capítulo 4

OA	K	Tema
FL-4.1.1	K2	Resumir las categorías de técnicas de prueba y sus subcategorías
FL-4.2.1	K3	Aplicar la partición de equivalencia para derivar casos de prueba en un escenario dado
FL-4.2.2	K3	Aplicar el análisis de valores límite para derivar casos de prueba en un escenario dado
FL-4.2.3	K3	Aplicar las pruebas de tabla de decisión para derivar casos de prueba en un escenario dado
FL-4.2.4	K3	Aplicar las pruebas de transición de estado para derivar casos de prueba en un escenario dado
FL-4.3.1	K2	Explicar las pruebas de sentencias
FL-4.3.2	K2	Explicar las pruebas de ramas
FL-4.3.3	K2	Explicar el valor de las pruebas de caja blanca
FL-4.4.1	K2	Explicar la suposición de errores
FL-4.4.2	K2	Explicar las pruebas exploratorias
FL-4.4.3	K2	Explicar las pruebas basadas en listas de verificación
FL-4.5.1	K2	Explicar los factores que hacen una buena historia de usuario (criterios INVEST)
FL-4.5.2	K2	Recordar los formatos típicos para redactar criterios de aceptación
FL-4.5.3	K2	Resumir las actividades del enfoque ATDD

Consejo para el examen: Las cuatro técnicas de caja negra (FL-4.2.1–4.2.4) son **K3** — el examen te dará un escenario y te pedirá que produzcas casos de prueba. Practica derivarlos, no solo explicarlos. Las técnicas de caja blanca (FL-4.3.1–4.3.2) son solo K2 — explicas la cobertura de sentencias y ramas, no la calculas.

4.1 Las Tres Categorías de Técnicas de Prueba — 💡 Comprender (K2)

Piensa en las técnicas de prueba como tres lentes diferentes para observar un sistema:

Categoría	También conocida como	Basada en	Característica clave
Caja negra	Basada en especificaciones	Comportamiento especificado	Las pruebas son independientes de la implementación — sobreviven a los cambios en el código
Caja blanca	Basada en estructura	Estructura interna/código	Las pruebas dependen de cómo está construido el software; se crean después del diseño/implementación
Basada en la experiencia	—	Conocimiento y habilidad del probador	Complementaria a las otras dos; encuentra defectos que las otras pasan por alto

Truco de memoria: Negra = no puedes ver dentro de la caja. Blanca = puedes ver todo dentro. Experiencia = ya has visto cajas similares romperse antes.

4.2 Técnicas de Prueba de Caja Negra — ⚙️ Aplicar (K3)

4.2.1 Partición de Equivalencia (PE) — ⚙️ Aplicar (K3)

La idea: Si el sistema trata todos los valores de un grupo de la misma manera, solo necesitas probar un valor de ese grupo.

- Divide los datos de entrada (o salida) en **particiones** donde todos los elementos se procesan de forma idéntica
- Un caso de prueba por partición es suficiente
- Partición válida:** contiene valores que el sistema debe aceptar
- Partición inválida:** contiene valores que el sistema debe rechazar
- 100% de cobertura PE** = ejercitar cada partición (válida E inválida) al menos una vez
- Cobertura = particiones ejercitadas ÷ total de particiones

Cuando existen múltiples entradas: Usa la **cobertura de cada elección** — al menos un valor de cada conjunto de particiones aparece en al menos un caso de prueba.

Ejemplo: Un campo acepta edades de 18 a 65. Particiones: menor de 18 (inválida), 18–65 (válida), mayor de 65 (inválida). Se necesitan al menos 3 casos de prueba — uno por partición.

4.2.2 Análisis de Valores Límite (AVL) — ⚙️ Aplicar (K3)

La idea: Los desarrolladores cometen más errores en los extremos de un rango — así que prueba ahí.

- Extiende la PE apuntando específicamente a los **límites** de las particiones ordenadas
- Solo aplicable a particiones **ordenadas** (donde existen valores mínimo/máximo)

Dos versiones:

Versión	Elementos de cobertura por límite	¿Más rigurosa?
AVL de 2 valores	Valor límite + su vecino más cercano de la partición adyacente	Menos
AVL de 3 valores	El valor límite en sí, el valor inmediatamente por debajo y el valor inmediatamente por encima	Más — puede detectar errores de uno en uno que el AVL de 2 valores pasa por alto

Ejemplo: Para un rango de 1 a 10 usando AVL de 3 valores, prueba: 0, 1, 2 (límite inferior) y 9, 10, 11 (límite superior).

100% de cobertura: Ejercitar todos los valores límite (2 valores) o todos los valores límite y sus vecinos (3 valores).

4.2.3 Pruebas de Tabla de Decisión — Aplicar (K3)

La idea: Cuando el sistema se comporta de manera diferente según combinaciones de condiciones, una tabla mapea cada combinación a su resultado esperado.

- Ideal para probar **reglas de negocio** y lógica compleja
- Cada **columna** = una regla de decisión (una combinación única de condiciones + acciones resultantes)
- Notación:
- **T** = la condición es verdadera, **F** = la condición es falsa
- **–** = la condición es irrelevante para el resultado
- **N/A** = la condición es inviable para esta regla
- **X** = la acción ocurre, **en blanco** = la acción no ocurre

100% de cobertura = ejercitar todas las columnas con combinaciones de condiciones viables. Cobertura = columnas viables ejercitadas ÷ total de columnas viables.

Nota: La cobertura se mide respecto a las columnas reales de la tabla utilizada. Si la tabla ha sido minimizada (reglas colapsadas), la cobertura se calcula respecto a sus columnas reales — no respecto al máximo teórico de 2^n .

Fortaleza: Revela brechas y contradicciones en los requisitos; es sistemática — nada queda sin considerar.

Debilidad: Con muchas condiciones, el número de reglas crece exponencialmente (2^n). Usa una tabla minimizada o un enfoque basado en riesgos para reducir el alcance.

4.2.4 Pruebas de Transición de Estado — Aplicar (K3)

La idea: Algunos sistemas se comportan de manera diferente según su estado actual. Esta técnica prueba esos comportamientos dependientes del estado.

Conceptos clave: - **Diagrama de estados:** muestra todos los estados posibles y las transiciones válidas entre ellos - Sintaxis de transición: **evento** [condición de guarda] / acción - **Tabla de estados:** equivalente a un diagrama de estados; filas = estados, columnas = eventos; celdas vacías = transiciones inválidas

Un caso de prueba = una secuencia de eventos que produce una secuencia de cambios de estado.

Tres criterios de cobertura (de más débil a más fuerte):

Cobertura	Qué ejercita	Notas
Todos los estados	Todos los estados al menos una vez	Más débil — puede lograrse sin probar todas las transiciones
Transiciones válidas (0-switch)	Todas las transiciones válidas — cada transición válida ejercitada al menos una vez, sin encadenar secuencias (0-switch = pasos individuales únicamente)	Criterio más ampliamente utilizado
Todas las transiciones	Todas las transiciones válidas + todas las transiciones inválidas	Más fuerte; requerido para software de misión crítica o seguridad crítica

Punto clave para el examen: Lograr la cobertura de transiciones válidas garantiza la cobertura de todos los estados. Lograr la cobertura de todas las transiciones garantiza ambas. Prueba solo **una** transición inválida por caso de prueba para evitar el enmascaramiento de defectos.

4.3 Técnicas de Prueba de Caja Blanca — 💡 Comprender (K2)

4.3.1 Pruebas de Sentencias — 💡 Comprender (K2)

- **Elementos de cobertura:** sentencias ejecutables
- Objetivo: diseñar casos de prueba para ejecutar sentencias hasta alcanzar un nivel de cobertura aceptable
- **100% de cobertura de sentencias** = cada sentencia ejecutable ejecutada al menos una vez
- Cobertura = sentencias ejercitadas ÷ total de sentencias ejecutables

Limitación: El 100% de cobertura de sentencias NO garantiza que todas las ramas estén probadas. Un defecto que solo se activa bajo una condición específica puede pasar desapercibido.

4.3.2 Pruebas de Ramas — 💡 Comprender (K2)

- **Elementos de cobertura:** ramas — transferencias de control entre nodos en un grafo de flujo de control
- Las ramas pueden ser **incondicionales** (código en línea recta) o **condicionales** (decisiones de if/else, switch, bucle)
- **100% de cobertura de ramas** = todas las ramas ejercitadas
- Cobertura = ramas ejercitadas ÷ total de ramas

Regla crítica: La cobertura de ramas subsume la cobertura de sentencias. Si logras el 100% de cobertura de ramas, automáticamente logras el 100% de cobertura de sentencias. Lo contrario NO es verdad.

Limitación: No detecta defectos que solo aparecen en un camino específico a través del código.

4.3.3 El Valor de las Pruebas de Caja Blanca — 💡 Comprender (K2)

Fortaleza: Tiene en cuenta toda la implementación — puede encontrar defectos incluso cuando la especificación es vaga, está desactualizada o es incompleta.

Debilidad: Si al software le falta completamente una funcionalidad requerida (defecto de omisión), las pruebas de caja blanca no lo detectarán — solo prueba lo que existe.

También útil en el análisis estático: El *análisis* de caja blanca (una forma de prueba estática) puede aplicarse a pseudocódigo o código no ejecutable para revisar la estructura sin ejecutarlo. Las *pruebas* de caja blanca (dinámicas) requieren código ejecutable — no confundas ambos.

Las pruebas de caja negra por sí solas no ofrecen ninguna medida de la cobertura real del código. Las pruebas de caja blanca proporcionan esa medición objetiva.

4.4 Técnicas de Prueba Basadas en la Experiencia — 💡 Comprender (K2)

4.4.1 Suposición de Errores — 💡 Comprender (K2)

La idea: Usa lo que sabes sobre cómo falla el software para anticipar dónde fallará.

Basada en el conocimiento del probador sobre: - Cómo ha funcionado (o fallado) la aplicación en el pasado - Tipos de errores que los desarrolladores cometen habitualmente - Tipos de fallos comunes en aplicaciones similares

Ataques de fallos: Crea una lista estructurada de errores, defectos y fallos probables → diseña pruebas específicamente para exponerlos. Las listas pueden provenir de la experiencia, datos de defectos o conocimiento de fallos comunes.

4.4.2 Pruebas Exploratorias — 💡 Comprender (K2)

La idea: Diseña, ejecuta y evalúa pruebas simultáneamente mientras aprendes sobre el sistema.

- Las pruebas se usan para aprender, explorar más profundamente y encontrar áreas no probadas
- **Enfoque basado en sesiones:** pruebas dentro de un intervalo de tiempo definido, guiadas por un **acta de prueba** que contiene los objetivos de prueba
- Cada sesión termina con una **reunión informativa** con los interesados

Más útil cuando: - Las especificaciones son escasas, inadecuadas o inexistentes - Existe una presión de tiempo significativa - Se desea complementar técnicas más formales

Más efectivo cuando el probador: tiene experiencia, conocimiento del dominio y aporta pensamiento analítico, curiosidad y creatividad.

4.4.3 Pruebas Basadas en Listas de Verificación — 💡 Comprender (K2)

La idea: Usa una lista de verificación preparada de condiciones de prueba para guiar las pruebas — especialmente cuando no existen casos de prueba detallados.

- Los elementos suelen formularse como preguntas

- Se construyen a partir de la experiencia, el conocimiento de los usuarios y la comprensión de cómo falla el software
- Deben **actualizarse regularmente** en función del análisis de defectos — las listas de verificación pueden desgastarse igual que las pruebas

NO incluir en las listas de verificación (*orientación del sector — no listada explícitamente en el plan de estudios CTFL 4.0*): - Elementos que pueden verificarse automáticamente - Elementos más adecuados como criterios de entrada/salida - Elementos demasiado generales

Compromiso: Las listas de verificación de alto nivel proporcionan consistencia y amplitud de cobertura, pero menos repetibilidad que los casos de prueba detallados.

4.5 Enfoques de Prueba Basados en la Colaboración —💡 Comprender (K2)

A diferencia de las otras técnicas (que se centran en *encontrar* defectos), los enfoques basados en la colaboración se enfocan en **evitar defectos mediante la comprensión compartida**.

4.5.1 Redacción Colaborativa de Historias de Usuario —💡 Comprender (K2)

Una **historia de usuario** representa una funcionalidad valiosa para un usuario o comprador. Cada historia de usuario tiene las **3 C** (Jeffries 2000):

C	Qué es
Card (Tarjeta)	El medio que describe la historia (tarjeta de índice, entrada en un tablero electrónico)
Conversation (Conversación)	Explica cómo se usará el software (verbal o documentada)
Confirmation (Confirmación)	Los criterios de aceptación

Formato estándar: *"Como [rol], quiero [objetivo], para poder [valor de negocio resultante]."*

Las buenas historias de usuario siguen INVEST: - Independiente - Negociable - Valiosa - Estimable - Small (pequeña) - Testable (comprobable)

Si un interesado no puede imaginar cómo probar una historia de usuario, es una señal de que la historia no está clara o no es suficientemente valiosa.

4.5.2 Criterios de Aceptación —💡 Comprender (K2)

Los criterios de aceptación son las **condiciones que una historia de usuario debe cumplir para ser aceptada** por los interesados. Son esencialmente las condiciones de prueba para las pruebas de aceptación.

Se usan para: - Definir el alcance de la historia de usuario - Alcanzar consenso entre los interesados - Describir tanto escenarios positivos como negativos - Servir como base para las pruebas de aceptación - Permitir una planificación y estimación precisas

Dos formatos más comunes:

Formato	Ejemplo
Orientado a escenarios (BDD)	Dado [contexto], Cuando [acción], Entonces [resultado]
Orientado a reglas	Lista de verificación de puntos de verificación o tabla de mapeo entrada-salida

4.5.3 Desarrollo Dirigido por Pruebas de Aceptación (ATDD) — 💡 Comprender (K2)

La idea: Escribe los casos de prueba *antes* de implementar la historia de usuario, usando los criterios de aceptación como guía.

Cómo funciona: 1. **Taller de especificación:** el equipo (clientes + desarrolladores + probadores) analiza conjuntamente la historia de usuario y los criterios de aceptación; resuelve ambigüedades y brechas 2.

Creación de casos de prueba: los casos de prueba se crean en base a los criterios de aceptación; sirven como ejemplos del comportamiento correcto 3. **Implementación:** los desarrolladores escriben código para pasar los casos de prueba

Orden de los casos de prueba: positivos (camino feliz) → negativos → características de calidad no funcionales (p. ej., rendimiento, usabilidad)

Los casos de prueba deben expresarse en un lenguaje **comprensible para los interesados** (lenguaje natural: precondiciones, entradas, postcondiciones).

Cuando se capturan en un formato de marco de automatización, las pruebas de aceptación se convierten en **requisitos ejecutables**.

Término	Definición
Técnica de caja negra	Deriva pruebas del comportamiento especificado; independiente de la estructura interna
Técnica de caja blanca	Deriva pruebas de la estructura interna/código
Técnica basada en la experiencia	Usa el conocimiento y la experiencia del probador para diseñar pruebas
Partición de equivalencia	Divide los datos en particiones procesadas de forma idéntica; una prueba por partición
Análisis de valores límite	Prueba en los extremos de las particiones de equivalencia
Pruebas de tabla de decisión	Prueba combinaciones de condiciones y sus acciones resultantes
Pruebas de transición de estado	Prueba el comportamiento basado en los estados del sistema y las transiciones entre ellos
Cobertura de sentencias	% de sentencias ejecutables ejercitadas por las pruebas
Cobertura de ramas	% de ramas ejercitadas; subsume la cobertura de sentencias
Suposición de errores	Anticipar defectos basándose en la experiencia y el conocimiento de fallos
Pruebas exploratorias	Diseño, ejecución y evaluación simultáneos dentro de una sesión con intervalo de tiempo definido
Pruebas basadas en listas de verificación	Pruebas guiadas por una lista preparada de condiciones, frecuentemente como preguntas
Historia de usuario	Descripción de una funcionalidad valiosa para un usuario; tiene Tarjeta, Conversación, Confirmación
INVEST	Criterios para una buena historia de usuario: Independiente, Negociable, Valiosa, Estimable, Small (pequeña), Testable (comprobable)
Criterios de aceptación	Condiciones que una historia de usuario debe cumplir para ser aceptada; condiciones de prueba para las pruebas de aceptación
ATDD	Casos de prueba escritos antes de implementar la historia de usuario, basados en los criterios de aceptación

Preguntas de Práctica

- ¿Cuáles son las tres categorías de técnicas de prueba y en qué se diferencian?
- En la partición de equivalencia, ¿por qué se considera suficiente una prueba por partición?
- ¿Cuál es la diferencia entre el AVL de 2 valores y el AVL de 3 valores? ¿Cuándo usarías cada uno?
- ¿Qué significa cada símbolo en una tabla de decisión (T, F, –, N/A, X)?
- ¿Cuáles son los tres criterios de cobertura de transición de estado, de más débil a más fuerte?
- ¿Por qué debes probar solo una transición inválida por caso de prueba en las pruebas de transición de estado?

- ¿Qué significa que "la cobertura de ramas subsume la cobertura de sentencias"?
- ¿Cuál es la principal fortaleza y debilidad de las pruebas de caja blanca?
- ¿Qué es un ataque de fallos en la suposición de errores?
- ¿Cuándo son más útiles las pruebas exploratorias?
- ¿Cuáles son las 3 C de una historia de usuario?
- ¿Qué significa INVEST?
- ¿Cuáles son los dos formatos más comunes para redactar criterios de aceptación?
- ¿Qué ocurre en el taller de especificación en ATDD?
- ¿Cuál es la diferencia principal entre las técnicas de detección de defectos (4.2–4.4) y los enfoques basados en la colaboración (4.5)?

Capítulo 5 - Gestión de las Actividades de Prueba

La idea principal: Las pruebas de calidad no ocurren por azar — deben planificarse, seguirse y cerrarse adecuadamente. Este capítulo trata el lado organizacional y de gestión de las pruebas: cómo definir la estrategia, estimar el esfuerzo, medir el avance, gestionar los riesgos y reportar los resultados.

Niveles Cognitivos en Este Capítulo

K1 = Recordar · K2 = Comprender · K3 = Aplicar — véase el Capítulo 1 para la leyenda completa.

Objetivos de Aprendizaje del Capítulo 5

OA	K	Tema
FL-5.1.1	K2	Ilustrar el propósito y el contenido de un plan de prueba
FL-5.1.2	K1	Reconocer cómo un tester aporta valor a la planificación de iteraciones y versiones
FL-5.1.3	K2	Comparar los criterios de entrada y los criterios de salida
FL-5.1.4	K3	Aplicar técnicas de estimación para calcular el esfuerzo de prueba en un escenario definido
FL-5.1.5	K3	Aplicar la priorización de casos de prueba en un escenario dado
FL-5.1.6	K2	Recordar los conceptos de la pirámide de pruebas
FL-5.1.7	K2	Resumir los cuadrantes de prueba y su relación con los niveles y tipos de prueba
FL-5.2.1	K2	Explicar el propósito del seguimiento y el control de pruebas, y la diferencia entre ambos
FL-5.2.2	K2	Resumir las métricas de prueba utilizadas
FL-5.2.3	K2	Resumir el propósito y el contenido de los informes de prueba
FL-5.3.1	K2	Explicar cómo los riesgos influyen en el alcance de las pruebas
FL-5.3.2	K1	Recordar la identificación y la evaluación de riesgos
FL-5.4.1	K3	Aplicar un informe de defectos a un escenario dado
FL-5.4.2	K2	Resumir el contenido y el propósito de un informe de defectos

Consejo para el examen: Hay tres ítems K3 — estimación (FL-5.1.4), priorización (FL-5.1.5) e informes de defectos (FL-5.4.1). El examen presentará una situación y pedirá producir algo: una estimación, una lista priorizada o un informe de defectos. FL-5.1.2 y FL-5.3.2 son K1 — puro recuerdo.

5.1 Planificación de Pruebas — 💡 Comprender (K2)

Un **plan de prueba** documenta los medios y el cronograma para alcanzar los objetivos de prueba. Es un documento vivo — se actualiza a medida que el proyecto evoluciona.

Contenido típico de un plan de prueba — 💡 Comprender (K2)

Sección	Contenido
Contexto	Alcance, base de prueba, restricciones, entorno de prueba
Objetivos	Lo que las pruebas pretenden lograr
Partes interesadas	Roles, responsabilidades, comunicación
Enfoque de prueba	Estrategia, técnicas, tipos de prueba
Cronograma	Hitos, esfuerzo, plazos
Recursos	Personas, herramientas, entornos
Criterios de entrada y salida	Cuándo comenzar y cuándo detener las pruebas
Riesgos e incidencias	Riesgos conocidos y estrategias de mitigación
Métricas	Qué se medirá para hacer seguimiento del avance

5.2 Contribución del Tester a la Planificación de Iteraciones y Versiones — 🧠 Recordar (K1)

En contextos ágiles e iterativos, los testers participan activamente en:

- **Planificación de versiones** — estimar el esfuerzo para funcionalidades a través de múltiples iteraciones; analizar riesgos; definir el enfoque de prueba general
- **Planificación de iteraciones** — analizar las historias de usuario en el backlog; evaluar la testeabilidad; identificar criterios de aceptación; estimar el esfuerzo de prueba por historia; participar en la identificación y evaluación de riesgos

Los testers aportan una perspectiva de calidad que los desarrolladores y los dueños de producto pueden no tener de forma natural — cuestionando supuestos, detectando criterios de aceptación faltantes e identificando ambigüedades de forma temprana.

5.3 Criterios de Entrada y Criterios de Salida — 💡 Comprender (K2)

Criterios de entrada — cuándo COMENZAR las pruebas — 💡 Comprender (K2)

Condiciones que deben cumplirse antes de que comience una actividad de prueba. Probar sin que estas se satisfagan desperdicia esfuerzo.

Ejemplos: - El testware está disponible (casos de prueba, scripts de prueba, datos de prueba) - El entorno de prueba está listo y configurado - El objeto de prueba ha superado una prueba de humo

Criterios de salida — cuándo DETENER las pruebas — 💡 Comprender (K2)

Condiciones que determinan cuándo una actividad de prueba está suficientemente completa.

Ejemplos: - Las pruebas planificadas se han ejecutado - Se ha alcanzado el nivel de cobertura (sentencias, requisitos) - La densidad de defectos está por debajo del umbral definido - Se ha agotado el tiempo o el presupuesto (salida pragmática)

En entornos ágiles, los criterios de entrada/salida suelen denominarse **Definition of Ready (DoR)** y **Definition of Done (DoD)**.

5.4 Técnicas de Estimación — ⚙️ Aplicar (K3)

Estimar cuánto esfuerzo de prueba se necesita es notoriamente difícil. ISTQB cubre dos técnicas principales:

Estimación Basada en Métricas — ⚙️ Aplicar (K3)

Utiliza datos de proyectos similares anteriores para estimar el esfuerzo actual.

- Derivar estimaciones a partir de métricas reales: tasas de defectos, tiempos de ejecución de pruebas, número de casos de prueba por punto de función
- Útil cuando se dispone de datos históricos y el proyecto actual es lo suficientemente similar

Estimación Basada en Expertos — ⚙️ Aplicar (K3)

Utiliza el conocimiento y el juicio de testers con experiencia.

Enfoques comunes: - **Planning Poker:** los miembros del equipo revelan simultáneamente sus estimaciones de esfuerzo; los valores atípicos se discuten hasta llegar a un consenso (usado en entornos ágiles) -

Wideband Delphi: rondas estructuradas de estimación anónima y discusión hasta que el equipo converge *(de versiones anteriores de ISTQB — CTFL 4.0 solo nombra Planning Poker como ejemplo de estimación basada en expertos)*

Ambas técnicas pueden combinarse — usar datos históricos como línea base y el juicio experto para ajustar las diferencias conocidas.

5.5 Priorización de Casos de Prueba — ⚙️ Aplicar (K3)

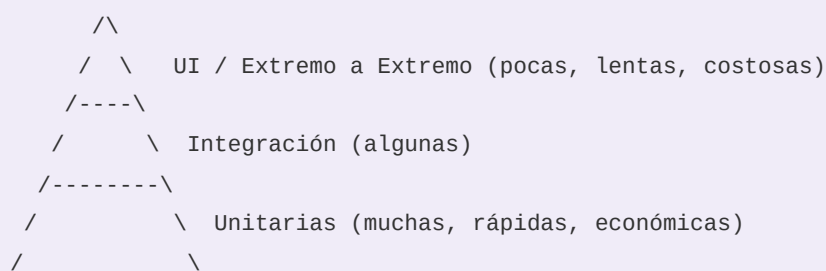
No todas las pruebas son igualmente importantes. La priorización determina el orden en que se ejecutan las pruebas.

Estrategia	Priorizar por
Basada en riesgo	Las áreas de mayor riesgo primero — mayor impacto si fallan
Basada en cobertura	Las pruebas que cubren más requisitos o código primero
Basada en requisitos	Las pruebas vinculadas a los requisitos más críticos o importantes
Basada en actividad	Orden basado en dependencias (algunas pruebas deben ejecutarse antes que otras) (concepto de apoyo — no definido en el programa de CTFL 4.0)

En la práctica, la priorización basada en riesgo es la más común y la más efectiva — maximiza la detección de defectos en el menor tiempo cuando el tiempo es limitado.

5.6 La Pirámide de Pruebas — Comprender (K2)

La **pirámide de pruebas** es un modelo que agrupa las pruebas por granularidad y recomienda cómo distribuirlas:



- **Base (unitarias):** muchas pruebas pequeñas, rápidas y aisladas — económicas de escribir y mantener
- **Medio (integración):** pruebas de interacciones entre componentes
- **Cima (UI/E2E):** pocas pruebas, lentas y frágiles — usar solo donde sea necesario

La pirámide implica: **automatizar al nivel más bajo posible**. Cuanto más se sube la automatización al nivel de la UI, más lenta y frágil se vuelve la suite de pruebas.

5.7 Cuadrantes de Prueba — Comprender (K2)

Los **cuadrantes de prueba** (Brian Marick) categorizan los tipos de prueba según dos ejes:

- **Orientado al negocio vs. Orientado a la tecnología**
- **Apoyando al equipo (guiando el desarrollo) vs. Criticando el producto (encontrando problemas)**

	Apoyando al equipo	Criticando el producto
Orientado a la tecnología	Q1: Pruebas de componente, pruebas unitarias, pruebas de integración	Q4: Pruebas de rendimiento, seguridad, fiabilidad y pruebas de "ilities"
Orientado al negocio	Q2: Pruebas funcionales automatizadas, pruebas de historia, prototipos, simulaciones	Q3: Pruebas exploratorias, pruebas de usabilidad, UAT, alfa/beta

Los cuadrantes son una **herramienta de comunicación** — ayudan a los equipos a hablar sobre qué tipos de pruebas son necesarios y quién debe realizarlos. No son prescriptivos.

5.8 Riesgo y Pruebas — 💡 Comprender (K2)

El riesgo es central en la planificación de pruebas. Orienta la priorización, las decisiones de cobertura y la profundidad de las pruebas aplicadas a cada área.

Conceptos clave — 🧠 Recordar (K1)

- **Riesgo**: la posibilidad de que ocurra un evento que cause un resultado adverso
- **Nivel de riesgo**: una combinación de **probabilidad** (likelihood) e **impacto** (consecuencia)
- **Riesgo de producto**: riesgo de que algo en el sistema falle — determina el alcance de las pruebas
- **Riesgo de proyecto**: riesgo para el proyecto en sí (cronograma, presupuesto, equipo) — se gestiona mediante la gestión de proyectos

Pruebas Basadas en Riesgo — 💡 Comprender (K2)

Usa el riesgo como guía principal para determinar: - **Qué** probar (las áreas de mayor riesgo reciben más atención) - **Cuánto** probar (las áreas más riesgosas reciben más casos de prueba) - **Cuándo** probar (los ítems de mayor riesgo se prueban antes)

Dos etapas: 1. **Identificación de riesgos**: sesión de lluvia de ideas con las partes interesadas — ¿qué podría salir mal? 2. **Evaluación de riesgos**: evaluar la probabilidad y el impacto; priorizar en consecuencia

Si un área de alto riesgo recibe pruebas adecuadas y no se encuentran defectos, esto aporta evidencia de calidad — no solo la ausencia de fallas.

5.9 Seguimiento de Pruebas, Control de Pruebas y Cierre de Pruebas — 💡 Comprender (K2)

Seguimiento de Pruebas — 💡 Comprender (K2)

Actividad continua de verificar el avance de las pruebas con respecto al plan.

Métricas de prueba comunes recopiladas:

Métrica	Lo que indica
% de casos de prueba ejecutados	Qué parte del plan está completada
% aprobados / fallidos / bloqueados	Señal de calidad
Tasa de detección de defectos	Cuántos defectos se están encontrando a lo largo del tiempo
Densidad de defectos por módulo	Dónde se concentran los defectos
Cobertura de requisitos / código	Con qué profundidad se ejercita el sistema
Consumo de recursos / presupuesto	¿El proyecto está en curso?

Control de Pruebas — 💡 Comprender (K2)

Tomar acciones correctivas cuando el seguimiento revela desviaciones respecto al plan.

Ejemplos de acciones de control: - Repriorizar las pruebas cuando se encuentra un defecto crítico - Ajustar el cronograma o el alcance si los riesgos se materializan - Cambiar el enfoque de prueba (p. ej., añadir sesiones exploratorias) - Agregar recursos si el equipo está retrasado

El seguimiento sin control es solo observación. El control es lo que hace útil al seguimiento.

Informes de Avance de Pruebas — 💡 Comprender (K2)

Comunican el estado actual a las partes interesadas. Típicamente incluyen: - Período de prueba cubierto - Avance con respecto al plan - Impedimentos y bloqueos - Actividades planificadas para el próximo período

Informes de Cierre de Pruebas — 💡 Comprender (K2)

Se producen al final de una fase de prueba, iteración o proyecto. Resumen: - Qué se probó, cómo y con qué resultados - Desviaciones con respecto al plan - Defectos encontrados y su estado - Testware a archivar o entregar - Lecciones aprendidas

5.10 Gestión de la Configuración en las Pruebas — 💡 Comprender (K2)

Alcance del examen: La gestión de la configuración es contexto de apoyo para FL-5.1.1 (contenido del plan de prueba) — no es un OA independiente en CTFL 4.0. Comprende su propósito y cómo se relaciona con la trazabilidad del testware, pero no memorices esta sección en profundidad.

La gestión de la configuración (GC) garantiza que el testware sea consistente, identificable y trazable.

Lo que la GC hace por las pruebas: - Identifica y rastrea versiones del testware (casos de prueba, scripts, entornos, datos) - Garantiza que los casos de prueba estén vinculados a la versión correcta del objeto de prueba - Habilita la reproducibilidad — ejecutar la misma prueba en el mismo entorno - Soporta el control de cambios — si el software cambia, ¿qué pruebas deben actualizarse?

En la práctica: los casos de prueba deben referenciar versiones específicas del software. "Probado con v2.3.1" tiene significado solo si la GC garantiza que la v2.3.1 puede reproducirse.

5.11 Gestión de Defectos — Aplicar (K3)

El proceso de gestión de defectos rastrea los defectos desde su descubrimiento hasta su cierre.

Ciclo de vida de un defecto (típico) — Comprender (K2)

Nota: El programa de CTFL 4.0 no prescribe estados de ciclo de vida específicos. El diagrama a continuación es un patrón de industria común para ilustración. En el examen, enfócate en los **campos del informe de defectos** (FL-5.4.1/5.4.2), no en los nombres de los estados del ciclo de vida.

Nuevo → Asignado → Abierto → Corregido → Re-prueba → Cerrado
 ↘ Rechazado / Diferido

Un buen informe de defectos incluye — Aplicar (K3)

Campo	Propósito
ID único	Seguimiento
Resumen	Descripción breve de la falla
Fecha y autor	Quién lo encontró y cuándo
Entorno	SO, navegador, versión, configuración
Pasos para reproducir	Secuencia exacta para reproducir la falla
Resultado esperado	Lo que debería haber ocurrido
Resultado real	Lo que realmente ocurrió
Severidad	Impacto en el sistema (Crítico, Mayor, Menor, Trivial)
Prioridad	Urgencia con la que debe corregirse
Estado	Estado actual en el ciclo de vida
Adjuntos	Registros de log, capturas de pantalla, videos

Severidad vs. Prioridad: Un error tipográfico en el texto de un botón de inicio de sesión tiene baja severidad pero posiblemente alta prioridad (es lo primero que ve cada usuario). Un fallo en una función poco usada podría tener alta severidad pero baja prioridad.

Término	Definición
Plan de prueba	Documento que describe el alcance, el enfoque, el cronograma y los recursos para las pruebas
Criterios de entrada	Condiciones que deben cumplirse antes de que comience una actividad de prueba
Criterios de salida	Condiciones que determinan cuándo una actividad de prueba está completa
Riesgo	La posibilidad de un evento con una consecuencia adversa
Riesgo de producto	Riesgo de que un componente del sistema no cumpla el comportamiento esperado
Riesgo de proyecto	Riesgo para el cronograma, el presupuesto o los recursos del proyecto
Pruebas basadas en riesgo	Usar el riesgo como guía para priorizar qué, cuánto y cuándo probar
Pirámide de pruebas	Modelo que recomienda más pruebas unitarias que de integración o de UI
Cuadrantes de prueba	Marco que categoriza las pruebas por audiencia (negocio vs. tecnología) y propósito (guiar vs. criticar)
Seguimiento de pruebas	Verificar el avance de las pruebas con respecto al plan
Control de pruebas	Tomar acciones correctivas cuando se encuentran desviaciones
Informe de defectos	Documento que captura una falla y toda la información necesaria para reproducirla, rastrearla y resolverla
Severidad	El impacto de un defecto en el sistema
Prioridad	La urgencia con la que debe corregirse un defecto
Gestión de la configuración	Rastreo de versiones del testware para garantizar consistencia y reproducibilidad

Preguntas de Práctica

- ¿Cuál es el propósito de un plan de prueba y qué incluye típicamente?
- ¿Cuál es la diferencia entre criterios de entrada y criterios de salida? Da un ejemplo de cada uno.
- ¿Cuáles son las dos técnicas de estimación principales cubiertas en ISTQB? ¿En qué se diferencian?
- ¿Qué es la priorización de pruebas basada en riesgo y por qué es la estrategia más común?
- ¿Qué es la pirámide de pruebas y qué recomienda sobre la automatización?
- ¿Cuáles son los cuatro cuadrantes de prueba y qué ejes los definen?
- ¿Cuál es la diferencia entre riesgo de producto y riesgo de proyecto?
- ¿Cuáles son las dos etapas de las pruebas basadas en riesgo (identificación y evaluación)?
- ¿Cuál es la diferencia entre seguimiento de pruebas y control de pruebas?
- ¿Qué debe incluir un informe de cierre de pruebas?
- ¿Cuál es la diferencia entre severidad y prioridad en la gestión de defectos?
- ¿Por qué es importante la gestión de la configuración para las pruebas?

- ¿Cómo se relacionan los criterios de entrada/salida con la Definition of Ready y la Definition of Done en entornos ágiles?
- ¿Qué métricas recopilarías para hacer seguimiento del avance de las pruebas?

Capítulo 6 - Herramientas de Prueba

La idea principal: La herramienta adecuada hace que las pruebas sean más rápidas, más consistentes y más exhaustivas — pero las herramientas no son una solución mágica. Saber qué categorías de herramientas existen, qué hacen realmente y cómo elegir las con criterio es de lo que trata este capítulo.

Niveles Cognitivos en Este Capítulo

K1 = Recordar · K2 = Comprender · K3 = Aplicar — véase el Capítulo 1 para la leyenda completa.

Objetivos de Aprendizaje del Capítulo 6

OA	K	Tema
FL-6.1.1	K2	Explicar cómo los diferentes tipos de herramientas de prueba apoyan las pruebas
FL-6.2.1	K1	Recordar los beneficios y los riesgos de la automatización de pruebas
FL-6.2.2	K1	Recordar las consideraciones para seleccionar una herramienta

Consejo para el examen: El Capítulo 6 es el capítulo más corto y **no tiene K3**. El nivel más alto es K2 (FL-6.1.1). FL-6.2.1 y FL-6.2.2 son K1 — recuerdo, no explicación. No inviertas demasiado tiempo aquí.

6.1 Soporte de Herramientas para las Pruebas — Comprender (K2)

Las herramientas de prueba pueden apoyar casi cualquier actividad de prueba. Se categorizan por la **actividad que soportan**, no por cómo se llaman — muchas herramientas modernas cubren múltiples categorías.

Punto clave del examen (FL-6.1.1): Debes ser capaz de relacionar una categoría de herramienta con la actividad de prueba que soporta. El formato de la pregunta suele ser: "¿qué tipo de herramienta soportaría la actividad X?"

Herramientas de Gestión e Informes — 💡 Comprender (K2)

Tipo de herramienta	Lo que soporta
Herramientas de gestión de pruebas	Planificación, seguimiento y control de actividades de prueba; gestión de casos de prueba, suites de prueba y cronogramas de ejecución
Herramientas de seguimiento de defectos	Registro, seguimiento y gestión de defectos a través de su ciclo de vida
Herramientas de gestión de la configuración	Rastreo de versiones del software bajo prueba y del testware; garantía de reproducibilidad
Herramientas de gestión de requisitos	Trazabilidad entre requisitos y casos de prueba

Herramientas de Prueba Estática — 💡 Comprender (K2)

Tipo de herramienta	Lo que soporta
Herramientas de análisis estático	Análisis del código fuente sin ejecutarlo — detección de violaciones a estándares de codificación, vulnerabilidades de seguridad, código inalcanzable, variables no definidas
Correctores ortográficos y gramaticales	Revisión de productos de trabajo escritos (requisitos, casos de prueba) en cuanto a calidad lingüística

Las herramientas de análisis estático forman parte de las pruebas estáticas (Capítulo 3). Complementan las revisiones manuales y pueden integrarse en pipelines de CI/CD para dar retroalimentación inmediata a los desarrolladores.

Herramientas de Diseño e Implementación de Pruebas — 💡 Comprender (K2)

Tipo de herramienta	Lo que soporta
Herramientas de generación de datos de prueba	Creación automática de datos de entrada para las pruebas — útil para valores límite, conjuntos de datos grandes y datos de producción enmascarados
Herramientas de diseño de casos de prueba	Generación de casos de prueba a partir de modelos o especificaciones (utilizadas en pruebas basadas en modelos)
Herramientas de pruebas basadas en modelos	Derivación automática de casos de prueba a partir de un modelo de comportamiento del sistema

Tipo de herramienta	Lo que soporta
Herramientas de ejecución de pruebas (frameworks de automatización de pruebas)	Ejecución automática de scripts de prueba; comparación de resultados reales con los esperados; registro de resultados
Herramientas de medición de cobertura	Medición de cuánto del código o de los requisitos ha sido ejercitado por las pruebas (p. ej., cobertura de sentencias, cobertura de ramas)

Enfoques comunes de ejecución de pruebas:

- **Pruebas orientadas a datos:** el script de prueba está separado de los datos de prueba; los datos se proporcionan desde fuentes externas (hoja de cálculo, base de datos). El mismo script se ejecuta con muchos conjuntos de datos.
- **Pruebas orientadas a palabras clave:** la lógica de prueba se expresa mediante palabras clave legibles (p. ej., `Login`, `ClickButton`); menos técnico y más fácil de mantener para personas no programadoras.

Herramientas de Pruebas No Funcionales — 💡 Comprender (K2)

Tipo de herramienta	Lo que soporta
Herramientas de pruebas de rendimiento	Simulación de carga, medición de tiempos de respuesta, identificación de cuellos de botella (pruebas de carga, pruebas de estrés, pruebas de resistencia)
Herramientas de pruebas de seguridad	Análisis de vulnerabilidades, prueba de autenticación y autorización, soporte para pruebas de penetración
Herramientas de pruebas de accesibilidad	Verificación de que el sistema puede ser utilizado por personas con discapacidades

Herramientas de DevOps y Colaboración — 💡 Comprender (K2)

Tipo de herramienta	Lo que soporta
Herramientas de pipeline CI/CD	Disparo automático de la ejecución de pruebas en cada confirmación de código; integración de las pruebas en el proceso de compilación
Herramientas de control de versiones / SCM	Gestión de versiones del código fuente y del testware; habilitación de ramas y desarrollo paralelo
Herramientas de colaboración	Comunicación, documentación de pruebas compartida, coordinación de equipos remotos

Las herramientas de DevOps son lo que hace posible las pruebas continuas — conectan la confirmación del desarrollador con el resultado de la ejecución de pruebas sin intervención manual (conexión con el Capítulo 2: shift left, DevOps).

6.2 Beneficios y Riesgos de la Automatización de Pruebas — 🧠

Recordar (K1)

La automatización de pruebas significa usar herramientas para ejecutar pruebas y comparar los resultados reales con los esperados de forma automática. Tiene mayor valor para pruebas **repetitivas, estables y de alto volumen** — no para todo.

Beneficios de la Automatización de Pruebas — 🧠 Recordar (K1)

Beneficio	Por qué importa
Reduce el trabajo manual repetitivo	Libera a los testers para enfocarse en pruebas exploratorias y de mayor valor
Previene errores humanos simples	Un script se ejecuta exactamente de la misma manera cada vez
Evaluación objetiva	Las métricas de cobertura y los resultados se miden de forma consistente
Fácil acceso a información del estado de las pruebas	Los paneles de resultados y los registros se generan automáticamente
Tiempo de ejecución de pruebas reducido	Las pruebas pueden ejecutarse en paralelo, de noche o en cada confirmación
Más pruebas en el mismo tiempo	Las suites de regresión que tomarían días en forma manual pueden ejecutarse en horas

Riesgos de la Automatización de Pruebas — 🧠 Recordar (K1)

Riesgo	Qué puede salir mal
Expectativas poco realistas	Las herramientas no encuentran errores automáticamente — solo verifican lo que se les indica que verifiquen
Esfuerzo subestimado	Automatizar pruebas requiere tiempo significativo para configurar, programar y mantener
Usar la herramienta equivocada	No toda herramienta se adapta a cualquier stack tecnológico o tipo de prueba
Dependencia excesiva de la automatización	Las pruebas automatizadas no pueden reemplazar el juicio humano, especialmente para UX y pruebas exploratorias
Carga de mantenimiento	Cuando el sistema cambia, los scripts de prueba se rompen y deben actualizarse
Falsa sensación de seguridad	Que las pruebas automatizadas pasen no significa que el sistema esté libre de defectos

La idea clave: La automatización es un multiplicador — multiplica la eficiencia de las buenas pruebas, pero también multiplica los problemas de las malas pruebas. Las pruebas deficientes automatizadas a escala son peores que ninguna automatización.

Recordar (K1)

Elegir una herramienta no es solo una decisión técnica — depende de la organización, el proyecto y el equipo.

Factores a considerar — 🧠 Recordar (K1)

Factor	Qué evaluar
Madurez organizacional	¿Tiene el equipo las habilidades para usar y mantener la herramienta?
Tecnología del SUT	¿Es la herramienta compatible con el stack tecnológico del sistema (lenguaje, plataforma, navegador)?
Brechas en el proceso de prueba	¿Qué actividades carecen actualmente de soporte de herramientas? ¿Cuáles se beneficiarían más?
Licenciamiento y costo	Código abierto vs. comercial; pago único vs. suscripción; costo total de propiedad
Soporte del proveedor	¿Hay capacitación disponible? ¿El proveedor está activo y es receptivo?
Integración con herramientas existentes	¿Encaja en el pipeline de CI/CD actual, el sistema de gestión de pruebas y el rastreador de defectos?

Buenas prácticas para la selección de herramientas — 🧠 Recordar (K1)

- Realizar una **prueba de concepto**: probar la herramienta en una parte real del proyecto antes de comprometerse
- Evaluar la **capacidad de respuesta del proveedor** y la comunidad alrededor de la herramienta
- Evaluar los **requisitos de capacitación** — tener en cuenta el tiempo de adaptación
- Involucrar tanto a **testers como a desarrolladores** en la decisión de selección
- Preferir herramientas que **se integren bien** sobre herramientas que requieran reconstruir todo a su alrededor

Distinción importante: Seleccionar una herramienta no es lo mismo que implementar la automatización de pruebas. La selección de herramientas es la decisión; la implementación es el proyecto que sigue — con su propia planificación, riesgos y estimación de esfuerzo.

Término	Definición
Automatización de pruebas	Uso de herramientas para ejecutar pruebas y comparar los resultados reales con los esperados de forma automática
Framework de automatización de pruebas	Un conjunto de reglas, bibliotecas y herramientas que proporcionan la estructura para los scripts de prueba automatizados
Pruebas orientadas a datos	Un enfoque de automatización de pruebas donde los datos de prueba se almacenan externamente y se alimentan a los scripts; el mismo script se ejecuta con diferentes conjuntos de datos
Pruebas orientadas a palabras clave	Un enfoque de automatización de pruebas donde las acciones de prueba se expresan como palabras clave legibles, separando la lógica de prueba de la implementación
Herramienta de análisis estático	Una herramienta que analiza el código fuente sin ejecutarlo, detectando defectos y violaciones a estándares
Herramienta de gestión de pruebas	Una herramienta que apoya la planificación, el seguimiento, el control y la generación de informes sobre actividades de prueba
Herramienta de pruebas de rendimiento	Una herramienta que simula carga y mide los tiempos de respuesta y el comportamiento del sistema bajo estrés
Pruebas basadas en modelos	Un enfoque donde los casos de prueba se generan automáticamente a partir de un modelo de comportamiento del sistema
CI/CD	Integración Continua / Entrega Continua — una práctica de DevOps donde los cambios de código desencadenan compilaciones y pruebas automatizadas
Herramienta de medición de cobertura	Una herramienta que mide cuánto del código o de los requisitos es ejercitado por una suite de pruebas

Preguntas de Práctica

- ¿Cuáles son las principales categorías de herramientas de prueba? Da un ejemplo de lo que soporta cada una.
- ¿Cuál es la diferencia entre las pruebas orientadas a datos y las pruebas orientadas a palabras clave?
- Nombra tres beneficios y tres riesgos de la automatización de pruebas.
- ¿Por qué la automatización de pruebas requiere un esfuerzo de mantenimiento continuo?
- ¿Qué factores deben considerarse al seleccionar una herramienta de prueba?
- ¿Qué es una prueba de concepto en el contexto de la selección de herramientas y por qué se recomienda?
- ¿Por qué una suite de pruebas automatizadas que pasa no garantiza la calidad del software?
- ¿Cuál es la diferencia entre una herramienta de análisis estático y una herramienta de ejecución de pruebas?
- ¿Cómo se conectan las herramientas de CI/CD con el concepto de shift left del Capítulo 2?

- ¿Qué tipo de herramienta usarías para medir la cobertura de ramas después de ejecutar tu suite de pruebas?

ISTQB CTFL — Glosario

Todos los términos clave de los Capítulos 1–6, ordenados alfabéticamente. Cada entrada muestra el(los) capítulo(s) donde se evalúa y el nivel cognitivo.

Cómo usarlo: Lee el término, intenta recordar la definición y luego verifícala. Dedica tiempo extra a los términos K2 (necesitas *explicarlos*, no solo recordarlos) y a los términos K3 (necesitas *aplicarlos*).

A

Acta de prueba Ch4  K2 Documento utilizado en las pruebas exploratorias para definir los objetivos de una sesión de prueba con tiempo limitado. Guía la exploración sin sobre-restringirla.

Análisis de impacto Ch2  K2

El proceso de evaluar qué partes de un sistema se ven afectadas por un cambio propuesto. Se utiliza antes de las pruebas de mantenimiento para definir el alcance de las pruebas de regresión. Depende de una buena trazabilidad.

Análisis de valores límite (BVA) Ch4  K3 Técnica de caja negra que extiende la partición de equivalencia apuntando específicamente a los límites de las particiones ordenadas. Dos versiones: BVA de 2 valores (límite + vecino más cercano) y BVA de 3 valores (límite + valor inmediatamente inferior + valor inmediatamente superior). Aprovecha el hecho de que los desarrolladores cometen más errores en los extremos de los rangos.

Anomalía Ch3  K1 Cualquier resultado o condición que se desvía de las expectativas, descubierto durante una revisión. Puede o no ser un defecto — necesita ser investigado.

Aseguramiento de la Calidad (QA) Ch1  K1 Actividad orientada al proceso y de carácter preventivo que mejora el proceso de desarrollo para evitar que ocurran defectos. Distinto de las pruebas, que están orientadas al producto y son de carácter correctivo. El QA establece el buen proceso; las pruebas verifican el resultado.

ATDD — Desarrollo Dirigido por Pruebas de Aceptación Ch2 Ch4  K2 Un enfoque donde los casos de prueba se escriben antes de implementar la historia de usuario, basándose en los criterios de aceptación. Pasos: taller de especificación → creación de casos de prueba → implementación. Estrechamente relacionado con BDD.

B

Base de prueba Ch1  K1 Toda la información utilizada para derivar casos de prueba — requisitos, historias de usuario, diseños, código, evaluaciones de riesgo, etc. Una buena trazabilidad vincula los casos de prueba con la base de prueba.

BDD — Desarrollo Dirigido por Comportamiento Ch2  K1 Un enfoque de prueba primero donde las pruebas se escriben en lenguaje natural usando el formato Dado/Cuando/Entonces, haciéndolas comprensibles para todo el equipo, incluidas las partes interesadas no técnicas.

Rama   Una transferencia de control entre nodos en un grafo de flujo de control. Las ramas pueden ser incondicionales o condicionales (p. ej., de una sentencia if/else o switch).

C

Caja negra (técnica)   Técnica de prueba que deriva casos de prueba del comportamiento especificado del software — sin conocimiento de su estructura interna. Las pruebas sobreviven a los cambios de código porque están basadas en la especificación. Ver también: *Partición de Equivalencia*, *Análisis de Valores Límite*, *Pruebas con Tabla de Decisión*, *Pruebas de Transición de Estado*.

Caja negra (tipo de prueba)   Tipo de prueba donde las pruebas se basan en la especificación o el comportamiento especificado del sistema, sin conocimiento de la estructura interna. Se puede aplicar en cualquier nivel de prueba.

Caja blanca (técnica)   Técnica de prueba que deriva casos de prueba de la estructura interna del software — caminos de código, ramas, flujos de datos. Las pruebas dependen de la implementación. Incluye las pruebas de sentencias y las pruebas de ramas.

Caja blanca (tipo de prueba)   Tipo de prueba donde las pruebas se basan en la estructura interna o implementación del sistema. Se puede aplicar en cualquier nivel de prueba.

Caso de prueba   Ver: *Objeto de prueba*, *Base de prueba*.

Causa raíz   La razón fundamental subyacente por la que se cometió un error — p. ej., un requisito mal entendido, falta de capacitación o documentación poco clara. Corregir las causas raíz previene futuros defectos.

CI/CD — Integración Continua / Entrega Continua   Práctica de DevOps donde cada cambio de código activa automáticamente una compilación y una ejecución de pruebas. Integra las pruebas directamente en el flujo de desarrollo, permitiendo retroalimentación rápida y apoyando el desplazamiento hacia la izquierda. Las herramientas de CI/CD conectan los commits de los desarrolladores con los resultados de las pruebas sin intervención manual.

Cobertura de ramas   El porcentaje de ramas ejercidas por un conjunto de pruebas. El 100% de cobertura de ramas garantiza el 100% de cobertura de sentencias (subsunción), pero no al revés. Más fuerte que la cobertura de sentencias.

Cobertura de sentencias   El porcentaje de sentencias ejecutables ejercidas por un conjunto de pruebas. El 100% de cobertura de sentencias no garantiza que todas las ramas estén probadas. Más débil que la cobertura de ramas.

Criterio de cobertura   Una regla o objetivo utilizado para determinar si un conjunto de pruebas es suficiente. Ejemplos: todas las particiones ejercidas (PE), todas las ramas ejercidas (cobertura de ramas), todas las transiciones válidas ejercidas (transición de estado).

Criterios de aceptación   Las condiciones que una historia de usuario debe cumplir para ser aceptada por las partes interesadas. Definen el alcance de la historia, guían las pruebas de aceptación y se escriben típicamente en formato orientado a escenarios (Dado/Cuando/Entonces) u orientado a reglas.

Criterios de entrada   Condiciones que deben cumplirse antes de que comience una actividad de prueba. Evita esfuerzo desperdiciado en pruebas cuando el sistema no está listo. En Agile: Definición de Listo (DoR).

Criterios de salida   Condiciones que determinan cuándo una actividad de prueba está suficientemente completa. Pueden incluir objetivos de cobertura, umbrales de densidad de defectos, o

D

Defecto Ch1  K1 Un fallo o imperfección en un producto de trabajo (código, documento, diseño) introducido por un error humano. Cuando se ejecuta, un defecto puede causar un fallo — o puede que no, dependiendo de las condiciones.

Definición de Listo (DoR) Ch5  K2 Equivalente ágil de los criterios de entrada — las condiciones que deben cumplirse antes de que pueda comenzar el trabajo en una historia de usuario o iteración.

Definición de Terminado (DoD) Ch5  K2 Equivalente ágil de los criterios de salida — las condiciones que deben cumplirse para que una historia de usuario o iteración se considere completa.

Desplazamiento hacia la izquierda Ch2  K2 La práctica de mover las actividades de prueba más temprano en el SDLC — realizando análisis, diseño e incluso ejecución de pruebas lo antes posible para encontrar defectos cuando son más baratos de corregir.

E

Error Ch1  K1 Un error humano — por ejemplo, un desarrollador que malinterpreta un requisito. Un error introduce un defecto en un producto de trabajo.

Especificación de prueba basada en listas de verificación Ch4  K2 Técnica basada en la experiencia donde las pruebas se guían por una lista preparada de condiciones, frecuentemente formuladas como preguntas. Las listas de verificación deben actualizarse regularmente basándose en el análisis de defectos para evitar que pierdan eficacia.

F

Fallo Ch1  K1 El comportamiento incorrecto visible y observable de un sistema causado por la ejecución de un defecto. No todos los defectos causan fallos — algunos requieren condiciones específicas para activarse.

Fase de fallo Ch4  K2

Una forma estructurada de suposición de errores: el evaluador crea una lista de errores y defectos probables, luego diseña pruebas específicamente para exponerlos.

G

Gestión de defectos Ch5  K3 El proceso de rastrear los defectos desde su descubrimiento hasta su cierre. Implica registrar, clasificar, priorizar, corregir, re-probar y cerrar defectos. El examen evalúa el contenido de un informe de defecto (FL-5.4.1/5.4.2).

H

Herramienta de análisis estático    **K2** Evaluación asistida por herramientas de un producto de trabajo contra una sintaxis formal — sin ejecutarlo. Ejemplos: linters, analizadores de código. Encuentra problemas como variables no definidas, código inalcanzable e incumplimientos de estándares de codificación. En el Ch6, las herramientas de análisis estático son una categoría de herramientas nombrada que apoya las actividades de pruebas estáticas.

Herramienta de automatización de pruebas   **K1** Ver: *Marco de automatización de pruebas*.

Herramienta de cobertura   **K1** Una herramienta que mide qué tanto del código o de los requisitos ha sido ejercido por un conjunto de pruebas. Produce métricas objetivas como porcentajes de cobertura de sentencias o de ramas.

Herramienta de gestión de pruebas   **K1** Una herramienta que apoya la planificación, monitoreo, control e informe de las actividades de prueba. Típicamente gestiona casos de prueba, suites de prueba, cronogramas de ejecución de pruebas y seguimiento de defectos en un solo lugar.

Herramienta de pruebas de rendimiento   **K1** Una herramienta que simula carga de usuarios concurrentes en un sistema y mide tiempos de respuesta, rendimiento y consumo de recursos bajo estrés. Se usa para pruebas de carga, pruebas de estrés y pruebas de resistencia.

I

Informe de defecto   **K3** Un documento que registra un defecto, que contiene toda la información necesaria para reproducirlo, rastrearlo y resolverlo. Campos clave: ID único, resumen, fecha/autor, entorno, pasos para reproducir, resultado esperado, resultado real, severidad, prioridad, estado, adjuntos.

Informe de avance de pruebas   **K2** Un informe que comunica el estado actual de las pruebas a las partes interesadas. Cubre el período, el progreso frente al plan, los impedimentos y los próximos pasos planificados.

Informe de finalización de pruebas   **K2** Un informe producido al final de una fase de prueba o proyecto. Resume lo que se probó, los resultados, las desviaciones del plan, el estado de los defectos, el producto de trabajo a archivar y las lecciones aprendidas.

Inspección   **K1** El tipo de revisión más formal. Sigue un proceso completo y definido basado en ISO/IEC 20246. La detección máxima de anomalías es el objetivo principal; se recopilan métricas. El autor **no puede** actuar como líder de revisión ni como escribano — único en este tipo de revisión.

INVEST   **K2** Un acrónimo que define las características de una buena historia de usuario: Independiente, **N**egociable, **V**aliosa, **E**stimable, **P**equaña (Small), **V**erificable (Testable).

M

Marco de automatización de pruebas   **K1** Un conjunto de reglas, bibliotecas y herramientas que proporcionan la estructura y las convenciones para escribir, organizar y ejecutar scripts de prueba automatizados. Los ejemplos incluyen marcos basados en datos y basados en palabras clave.

Moderador   **K1** El rol responsable de conducir una reunión de revisión de manera efectiva. Gestiona la facilitación, la mediación y el tiempo, y garantiza un entorno seguro para la retroalimentación. Requerido en revisiones técnicas e inspecciones; opcional en recorridos.

N

Nivel de prueba   **K2** Un grupo de actividades de prueba organizadas y gestionadas en conjunto, con un enfoque específico y una base de prueba. Los cinco niveles: pruebas de componentes, pruebas de integración de componentes, pruebas de sistema, pruebas de integración de sistema y pruebas de aceptación.

O

OAT — Pruebas de Aceptación Operacional   **K2** Un tipo de pruebas de aceptación enfocado en la disponibilidad operacional: respaldo y recuperación, seguridad, rendimiento bajo carga y otras características operacionales.

Objeto de prueba   **K1** El producto de trabajo específico que se está probando — p. ej., un componente, un sistema, un documento o una configuración.

P

Parte interesada   **K1** Ver: *Criterios de aceptación, Riesgo*.

Partición de equivalencia (PE)   **K3** Técnica de caja negra que divide los datos de entrada (o salida) en particiones donde se espera que el sistema trate todos los valores de forma idéntica. Un caso de prueba por partición es suficiente. Cobertura = particiones ejercidas ÷ total de particiones.

Pirámide de pruebas   **K2** Un modelo que recomienda una distribución de pruebas entre niveles: muchas pruebas unitarias (rápidas, baratas), menos pruebas de integración y muy pocas pruebas de interfaz/extremo a extremo (lentas, frágiles). Fomenta la automatización en el nivel efectivo más bajo.

Plan de prueba   **K2** Un documento que describe los objetivos, el alcance, el enfoque, el cronograma y los recursos para un esfuerzo de prueba. Un documento vivo que se actualiza a medida que el proyecto evoluciona.

Prioridad   **K2** La urgencia con la que un defecto necesita ser corregido, desde una perspectiva de negocio. Alta prioridad significa que debe corregirse pronto — independientemente de la severidad. Un error cosmético en la pantalla de inicio de sesión puede ser de baja severidad pero alta prioridad.

Producto de trabajo   **K2** Ver: *Producto de prueba (Testware)*.

Producto de prueba (Testware)   **K2** Todos los artefactos producidos por las actividades de prueba — incluyendo planes de prueba, casos de prueba, scripts de prueba, datos de prueba, registros de prueba, informes de defectos e informes de finalización.

Pruebas basadas en experiencia   **K2** Una categoría de técnicas de prueba basadas en el conocimiento, la intuición y la experiencia del evaluador. Complementa las técnicas de caja negra y caja blanca. Incluye la suposición de errores, las pruebas exploratorias y las pruebas basadas en listas de verificación.

Pruebas basadas en modelos   **K1** Un enfoque de prueba donde los casos de prueba se derivan automáticamente de un modelo de comportamiento del sistema. El modelo (p. ej., una máquina de estados o

tabla de decisión) sirve como base de prueba; una herramienta genera casos de prueba a partir de él. Reduce el esfuerzo manual de diseño de pruebas para sistemas complejos.

Pruebas con tabla de decisión   **Ch4** Técnica de caja negra utilizada para probar combinaciones de condiciones y sus acciones resultantes. Cada columna de la tabla representa una regla (combinación única de condiciones). Más adecuada para probar lógica de negocio con comportamiento condicional complejo.

Pruebas con datos  **Ch6** **K1** Un enfoque de automatización de pruebas donde el script de prueba está separado de los datos de prueba. Los datos se almacenan externamente (p. ej., en una hoja de cálculo o base de datos) y se alimentan al script en tiempo de ejecución, permitiendo que el mismo script se ejecute con muchos conjuntos de datos diferentes.

Pruebas con palabras clave  **Ch6** **K1** Un enfoque de automatización de pruebas donde las acciones de prueba se expresan como palabras clave legibles (p. ej., `Iniciar sesión`, `HacerClick`, `VerificarTexto`). Las palabras clave se implementan por separado, permitiendo que personas sin conocimientos de programación escriban y mantengan pruebas sin conocimientos de scripting.

Pruebas de aceptación  **Ch2** **K2** El nivel de prueba enfocado en la validación — confirmar que el sistema está listo para su despliegue. Realizadas por representantes del negocio o usuarios finales. Incluye UAT, OAT, pruebas contractuales/regulatorias, pruebas alfa y pruebas beta.

Pruebas de componentes  **Ch2** **K2** El nivel de prueba enfocado en componentes individuales probados de forma aislada. También llamadas pruebas unitarias. Típicamente realizadas por desarrolladores.

Pruebas de confirmación  **Ch2** **K2** Pruebas realizadas después de que un defecto ha sido corregido, para verificar que ese defecto específico ya no causa un fallo. También llamadas re-pruebas.

Pruebas de integración de componentes  **Ch2** **K2** Un nivel de prueba enfocado en probar las interfaces e interacciones entre componentes integrados. Generalmente realizadas por desarrolladores o evaluadores después de las pruebas de componentes.

Pruebas de integración de sistema  **Ch2** **K2** Un nivel de prueba enfocado en las interfaces entre el sistema bajo prueba y los sistemas o servicios externos. Prueba las interacciones en el límite del sistema.

Pruebas de mantenimiento  **Ch2** **K2** Pruebas realizadas en un sistema que ya está en producción cuando necesita cambiar. Activadas por modificaciones (mejoras, correcciones de errores, hot fixes), actualizaciones/migraciones (nueva plataforma, SO, entorno) o retiro (archivado de datos al fin de vida útil).

Pruebas de rama  **Ch4** **K2** Técnica de caja blanca que diseña casos de prueba para ejercer todas las ramas en el código. Corresponde a la cobertura de ramas.

Pruebas de regresión  **Ch2** **K2** Re-ejecución de pruebas después de un cambio para detectar efectos secundarios no deseados — asegurando que la funcionalidad existente no se haya roto. Crece con cada versión y es un candidato sólido para la automatización.

Pruebas de sentencias  **Ch4** **K2** Técnica de caja blanca que diseña casos de prueba para ejercer sentencias ejecutables. Corresponde a la cobertura de sentencias.

Pruebas de sistema  **Ch2** **K2** Un nivel de prueba que evalúa el comportamiento de extremo a extremo de todo el sistema frente a sus requisitos especificados. Típicamente realizadas por evaluadores independientes.

Pruebas de transición de estado  **Ch4** **K3** Técnica de caja negra que prueba el comportamiento de un sistema basándose en sus estados y las transiciones entre ellos. Criterios de cobertura: todos los estados (más débil), todas las transiciones válidas / cobertura 0-switch (más común), todas las transiciones incluidas las inválidas (más fuerte).

Pruebas dinámicas  **Ch1** **Ch3** **K2** Pruebas realizadas ejecutando el software. Los defectos se revelan a través de fallos — el sistema debe ejecutarse. Contraste con las pruebas estáticas.

Pruebas estáticas     Pruebas realizadas sin ejecutar el software. Incluye tanto revisiones manuales como análisis estático automatizado. Puede encontrar defectos en productos de trabajo no ejecutables (requisitos, diseños, planes de prueba) que las pruebas dinámicas no pueden alcanzar.

Pruebas exploratorias    Técnica basada en la experiencia donde el evaluador simultáneamente diseña, ejecuta y evalúa pruebas mientras aprende sobre el sistema. Usa un *acta de prueba* para guiar cada sesión con tiempo limitado. Más efectiva cuando las especificaciones son incompletas o el tiempo es limitado.

Pruebas funcionales    Un tipo de prueba que evalúa *qué* hace el sistema — su completitud funcional, corrección e idoneidad. Se puede aplicar en cualquier nivel de prueba.

Pruebas no funcionales    Un tipo de prueba que evalúa *qué tan bien* funciona el sistema — cubriendo rendimiento, seguridad, usabilidad, fiabilidad, mantenibilidad, portabilidad y seguridad funcional. Se puede aplicar en cualquier nivel de prueba.

R

Recorrido    Un tipo de revisión conducida por el autor, quien guía a los participantes a través del producto de trabajo. Tiene como objetivo educar, lograr consenso y detectar anomalías. La revisión individual previa es opcional; el escribano es obligatorio.

Riesgo    La posibilidad de que ocurra un evento que cause una consecuencia adversa. Caracterizado por su probabilidad e impacto. La combinación de ambos determina el nivel de riesgo.

Riesgo de producto    El riesgo de que un componente o característica del sistema no cumpla con el comportamiento o la calidad esperados. Los riesgos de producto impulsan el alcance y la profundidad de las pruebas.

Riesgo de proyecto    El riesgo para el proyecto en sí — cronograma, presupuesto, recursos o equipo. Gestionado a través de la gestión de proyectos, no de las pruebas.

Revisión    Una forma de pruebas estáticas donde una o más personas evalúan manualmente un producto de trabajo. Los tipos van desde informal hasta formal (inspección). Objetivo: encontrar anomalías, mejorar la calidad, construir comprensión compartida.

Revisión informal    El tipo de revisión menos formal. Sin proceso definido, sin salida requerida. El objetivo es simplemente encontrar anomalías a través de una inspección informal.

Revisión técnica    Un tipo de revisión semiformal conducida por un moderador capacitado o un líder par con revisores técnicamente calificados. El enfoque está en lograr consenso sobre cuestiones técnicas y detectar anomalías.

S

SDLC — Ciclo de Vida del Desarrollo de Software    El proceso utilizado para planificar, crear, probar y entregar software. La elección del modelo de SDLC (secuencial, iterativo, incremental) impacta directamente en cómo se estructura las pruebas y cuándo ocurren.

Severidad    El impacto de un defecto en el sistema — qué tan gravemente afecta la funcionalidad, los datos o las operaciones. Clasificada como Crítica, Mayor, Menor o Trivial. No es lo mismo que prioridad.

Suposición de errores   **K2** Técnica basada en la experiencia que usa el conocimiento del evaluador sobre cómo falla típicamente el software para anticipar y apuntar a defectos probables. Puede estructurarse como un *ataque de fallos* usando una lista de errores probables.

Subsunción   **K2** Ver: *Cobertura de ramas*.

T

TDD — Desarrollo Dirigido por Pruebas   **K1** Una práctica de desarrollo donde los desarrolladores escriben primero una prueba unitaria fallida, luego escriben código para hacerla pasar, y luego refactorizan. Implementa el principio de pruebas tempranas.

Tipo de prueba   **K2** Una clasificación de las pruebas según qué aspecto del sistema se está probando: funcional, no funcional, caja negra o caja blanca. Los cuatro tipos se pueden aplicar en cualquier nivel de prueba.

Trazabilidad   **K2** Ver: *Base de prueba*, *Gestión de defectos*.

U

UAT — Pruebas de Aceptación de Usuario   **K2** Un tipo de pruebas de aceptación donde usuarios reales validan que el sistema satisface sus necesidades y está listo para su uso en producción.

V

Validación   **K1** Confirmar que un producto satisface las necesidades del usuario y otras partes interesadas. La pregunta es: "*¿Estamos construyendo el producto correcto?*" Se enfoca en la adecuación al propósito.

Verificación   **K1** Confirmar que un producto de trabajo cumple con sus requisitos especificados. La pregunta es: "*¿Estamos construyendo el producto correctamente?*" Se enfoca en la conformidad con las especificaciones.

Z

Cuadrantes de prueba   **K2** Un marco (Brian Marick) para categorizar los tipos de prueba mediante dos ejes: orientado a la tecnología vs. orientado al negocio, y apoyando al equipo vs. criticando el producto. Q1 (tecnología, apoyo): pruebas unitarias/de integración. Q2 (negocio, apoyo): pruebas funcionales/de historias. Q3 (negocio, crítica): pruebas exploratorias/UAT. Q4 (tecnología, crítica): rendimiento/seguridad.

Evaluación de riesgos   **K1** La segunda etapa de las pruebas basadas en riesgos: evaluar los riesgos identificados según su probabilidad e impacto para determinar qué áreas necesitan más atención en las pruebas.

Identificación de riesgos   **K1** La primera etapa de las pruebas basadas en riesgos: hacer una lluvia de ideas con las partes interesadas para identificar qué podría salir mal en el sistema.

Líder de revisión Ch3  K1 El rol responsable de organizar la revisión en general: decidir quién participa, cuándo y dónde. No puede ser la misma persona que el escribano o el autor en una inspección.

Escribano Ch3  K1 El rol responsable de registrar anomalías, decisiones y nuevos hallazgos durante una reunión de revisión. Obligatorio en recorridos e inspecciones.

Automatización de pruebas Ch6  K1 El uso de herramientas para ejecutar pruebas y comparar resultados reales con resultados esperados automáticamente, sin intervención manual. Más efectiva para pruebas repetitivas, estables y de alto volumen. No es un reemplazo del juicio humano y las pruebas exploratorias.

Pruebas basadas en riesgos Ch5  K2 Un enfoque de prueba que usa el riesgo como guía principal para decidir qué probar, cuánto probar y cuándo probar. Asegura que las áreas de mayor riesgo reciban mayor atención en las pruebas — especialmente importante cuando el tiempo es limitado.

Historia de usuario Ch4  K2 Una descripción de una característica que entrega valor a un usuario o comprador. Escrita como: "*Como [rol], quiero [objetivo], para que [valor de negocio].*" Tiene tres componentes: Tarjeta, Conversación, Confirmación (las 3 C). Debe seguir los criterios INVEST.

Pruebas alfa Ch2  K2 Un tipo de pruebas de aceptación realizadas por usuarios internos en las instalaciones del desarrollador, antes del lanzamiento.

Pruebas beta Ch2  K2 Un tipo de pruebas de aceptación realizadas por usuarios externos en su propio entorno, después del lanzamiento.

Gestión de la configuración (CM) Ch5  K2 La disciplina de identificar, rastrear y controlar las versiones del producto de prueba y los objetos de prueba. Garantiza que los casos de prueba puedan vincularse a la versión correcta del software y que las pruebas sean reproducibles. Contexto de apoyo para FL-5.1.1 — no es un objetivo de aprendizaje independiente.

Control de pruebas Ch5  K2 La actividad de tomar acciones correctivas cuando el monitoreo de pruebas revela desviaciones del plan de prueba — como reprioritizar pruebas, ajustar el alcance o agregar recursos.

Monitoreo de pruebas Ch5  K2 La actividad continua de verificar el progreso real de las pruebas frente al plan, usando métricas. Permite el control de pruebas cuando se detectan desviaciones.

ISTQB CTFL — Errores Comunes

Trampas típicas del examen, conceptos fácilmente confundidos y consejos para evitar caer en ellos.

Conceptos Fácilmente Confundidos

Error vs. Defecto vs. Fallo vs. Causa Raíz Ch1

Término	Qué es	Quién lo crea
Error	Un error humano	Desarrollador, analista, evaluador
Defecto	El fallo que el error introduce en el producto de trabajo	Causado por el error
Fallo	El comportamiento incorrecto visible cuando el defecto se ejecuta	Causado por el defecto
Causa raíz	La razón subyacente por la que se cometió el error	Problema de proceso o entorno

Trampa: "Un defecto siempre causa un fallo." — FALSO. Un defecto puede que nunca se ejecute, o puede que solo falle bajo condiciones específicas.

Pruebas vs. Aseguramiento de la Calidad Ch1

	Pruebas	QA
Enfoque	Producto	Proceso
Objetivo	Encontrar defectos	Prevenir defectos
Orientación	Correctiva	Preventiva

Trampa: "El QA y las pruebas son lo mismo." — FALSO. Las pruebas son parte del QA, pero el QA es más amplio — abarca todo el proceso de desarrollo.

Verificación vs. Validación Ch1

- Verificación** → "¿Estamos construyendo el producto **correctamente**?" — conformidad con la especificación
- Validación** → "¿Estamos construyendo el producto **correcto**?" — adecuación a las necesidades del usuario

Pruebas de Confirmación vs. Pruebas de Regresión Ch2

- **Pruebas de confirmación:** re-prueba un defecto específico que fue corregido — ¿funcionó *esta* corrección?
- **Pruebas de regresión:** re-prueba todo el sistema después de cualquier cambio — ¿se rompió *algo más*?

Trampa: "Las pruebas de regresión solo se necesitan después de correcciones de errores." — FALSO. Cualquier cambio (mejora, refactorización, cambio de configuración) puede introducir regresiones.

Caja Negra / Caja Blanca como Tipos de Prueba vs. Técnicas de Prueba Ch2 Ch4

Los mismos términos aparecen en dos capítulos diferentes con significados relacionados pero distintos:

Contexto	Significado
Ch2 — Tipos de prueba	<i>En qué</i> se basan las pruebas: especificación (caja negra) vs. estructura (caja blanca)
Ch4 — Técnicas de prueba	<i>Cómo</i> se derivan los casos de prueba: de la especificación (técnicas de caja negra) vs. del código (técnicas de caja blanca)

Trampa: Ver "caja negra" en una pregunta y asumir que siempre se refiere a la técnica del Ch4. Verifica de qué está hablando realmente la pregunta.

Pruebas Estáticas vs. Análisis Estático Ch3

- **Pruebas estáticas** = la categoría amplia — evaluar un producto de trabajo *sin ejecutarlo* (incluye revisiones Y análisis basado en herramientas)
- **Análisis estático** = el subconjunto basado en herramientas de las pruebas estáticas — análisis automatizado contra una sintaxis formal

Trampa: "El análisis estático y las pruebas estáticas son lo mismo." — FALSO. El análisis estático es *un tipo* de pruebas estáticas.

Recorrido vs. Revisión Técnica vs. Inspección ch3

	Recorrido	Revisión Técnica	Inspección
Conducida por	Autor	Moderador o líder par	Moderador (NO el autor)
Formalidad	Baja–Media	Media–Alta	La más alta
Escribano	Obligatorio	Depende	Obligatorio
¿Autor como líder?	Sí	No	Nunca

Trampa: "En un recorrido, la revisión individual previa es obligatoria." — FALSO. Es *opcional*. **Trampa:** "En una inspección, el autor puede liderar la revisión." — FALSO. El autor no puede ser el líder de revisión ni el escribano.

Partición de Equivalencia vs. Análisis de Valores Límite ch4

- **PE** — selecciona *un valor representativo* de cada partición (cualquier valor, no necesariamente el extremo)
- **BVA** — apunta específicamente a los *extremos* de las particiones ordenadas; extiende la PE, no la reemplaza

Trampa: "BVA reemplaza a PE." — FALSO. BVA extiende la PE añadiendo casos de prueba específicos de límites además de ella.

BVA de 2 Valores vs. BVA de 3 Valores ch4

- **2 valores:** el valor límite + su vecino más cercano de la partición adyacente
- **3 valores:** el propio valor límite + el valor inmediatamente inferior + el valor inmediatamente superior

Trampa: "El BVA de 3 valores prueba tres particiones." — FALSO. Prueba tres valores específicos alrededor de un límite.

Cobertura de Sentencias vs. Cobertura de Ramas ch4

- **Cobertura de sentencias:** cada sentencia ejecutable ejecutada al menos una vez
- **Cobertura de ramas:** cada rama (resultado verdadero/falso de cada decisión) ejercida

Regla crítica: La cobertura de ramas **subsume** la cobertura de sentencias — el 100% de cobertura de ramas garantiza el 100% de cobertura de sentencias. Lo inverso NO es verdad. **Trampa:** "El 100% de cobertura de sentencias significa que todas las ramas están probadas." — FALSO.

Severidad vs. Prioridad ch5

- **Severidad** = qué tan gravemente afecta el defecto al sistema (visión técnica)

- **Prioridad** = con qué urgencia necesita corregirse (visión de negocio)

Trampa: "Un defecto de alta severidad siempre es de alta prioridad." — FALSO. Ejemplo: un bloqueo en una función usada por el 0,1% de los usuarios = alta severidad, baja prioridad. Ejemplo: un error tipográfico en la página de inicio de sesión = baja severidad, alta prioridad.

Monitoreo de Pruebas vs. Control de Pruebas Ch5

- **Monitoreo** = recopilar datos y verificar el progreso frente al plan (observar)
- **Control** = tomar acciones correctivas cuando se detectan desviaciones (actuar)

Trampa: "El monitoreo y el control son la misma actividad." — FALSO. El monitoreo sin control es solo observación.

Riesgo de Producto vs. Riesgo de Proyecto Ch5

- **Riesgo de producto** = riesgo de que el sistema no cumpla con las expectativas de calidad → impulsa las decisiones de *prueba*
 - **Riesgo de proyecto** = riesgo para el cronograma, presupuesto o recursos del proyecto → gestionado por la *gestión de proyectos*
-

Trampas del Examen — Afirmaciones que Parecen Verdaderas Pero No lo Son

Afirmación trampa	Por qué está mal
"Las pruebas demuestran que el software es correcto."	Las pruebas muestran la <i>presencia</i> de defectos, nunca su ausencia (Principio 1)
"Con suficiente tiempo, podemos probar todo."	Las pruebas exhaustivas son imposibles (Principio 2)
"Si todas las pruebas pasan, el software no tiene defectos."	La ausencia de defectos es una falacia (Principio 7)
"Más pruebas siempre significan mayor calidad."	Las pruebas dependen del contexto; más no siempre es mejor (Principio 6)
"El autor puede liderar o ser escribano en una inspección."	No está permitido — regla única para las inspecciones
"Q1 en los cuadrantes de prueba está orientado al negocio."	Q1 está <i>orientado a la tecnología</i> + apoya al equipo; Q2 está orientado al negocio
"Las pruebas de regresión solo se ejecutan después de una corrección de error."	Cualquier cambio — mejoras, refactorizaciones, migraciones — puede romper el comportamiento existente
"El 100% de cobertura de sentencias garantiza pruebas completas."	No prueba todas las ramas; la cobertura de ramas es más fuerte
"Las pruebas exploratorias no tienen estructura."	Usan <i>sesiones</i> con tiempo limitado guiadas por un <i>acta de prueba</i>
"La cobertura 0-switch prueba secuencias de transiciones."	0-switch = transiciones individuales solo, sin encadenamiento
"Las pruebas de caja blanca son siempre dinámicas."	El <i>análisis</i> de caja blanca puede aplicarse estáticamente a pseudocódigo o código no ejecutable
"Wideband Delphi es una técnica nombrada del CTFL 4.0."	Solo el Planning Poker está nombrado; Wideband Delphi es de versiones anteriores
"Un recorrido requiere revisión individual previa."	La revisión individual es <i>opcional</i> en los recorridos; solo el escribano es obligatorio
"El BVA puede aplicarse a cualquier partición."	El BVA solo aplica a particiones <i>ordenadas</i> (aquellas con valores mínimos/máximos)

Consejos para Evitar las Trampas

Lee el enunciado de la pregunta con cuidado. Las preguntas del ISTQB a menudo dependen de una sola palabra: *MEJOR*, *MÁS apropiado*, *EXCEPTO*, *NO*, *MENOS probable*. Perder esa palabra invierte la respuesta correcta.

Conoce la diferencia entre los niveles K en la pregunta. - Una pregunta K1 te pide *identificar* o *recordar* — elige la definición correcta. - Una pregunta K2 te pide *explicar* o *distinguir* — elige la explicación más precisa. - Una pregunta K3 te da un *escenario* — aplica la técnica y deriva el resultado.

Cuando dos respuestas parecen correctas, ve a la más específica. El ISTQB prefiere la respuesta que coincide más precisamente con la definición del programa de estudios. La respuesta "casi correcta" generalmente es un distractor que intercambia una palabra clave.

Para preguntas de técnica K3, trabaja en papel. No intentes hacer PE, BVA o transición de estado en tu cabeza. Esboza rápidamente las particiones o la tabla — detectarás errores que perderías mentalmente.

La definición del programa de estudios prevalece sobre la práctica de la industria. Si llevas años trabajando en QA y algo te "parece" correcto, pero contradice la definición del programa de estudios CTFL 4.0, ve con el programa de estudios. El examen evalúa el estándar, no la variación del mundo real.

Presta atención a los absolutos. Palabras como *siempre*, *nunca*, *todo*, *solo*, *imposible* son frecuentemente trampas. Los principios de pruebas 1 y 2 abordan específicamente qué es y qué no es posible en las pruebas.

Cuando no estés seguro entre dos respuestas, elimina primero. Descarta las dos respuestas claramente incorrectas. Luego compara las dos restantes con la redacción exacta del programa de estudios — la respuesta correcta usará la terminología correcta.